

Department of Aeronautics

Imperial College London

Application of Multi-Scale Graph Neural Networks for Predicting Transient Dynamics: Cardiac Electrophysiology

Author:

Archie Dobson

Supervisor:

Chris Cantwell

A thesis submitted for the degree of

*MSc Advanced Computational Methods for Aeronautics, Flow Management and
Fluid-Structure Interaction*

November 15, 2025

Acknowledgements

Acknowledgements go here.

Contents

1	Introduction	1
1.1	Motivation and Aims	1
1.2	Cardiac Electrophysiology	1
1.2.1	The Microstructures of Cardiac Tissue	2
1.2.2	The Macrostructures of Cardiac Tissue	2
1.2.3	Acquiring a Patient Specific Model of the Heart	3
1.3	Mathematical Models of Cardiac Tissue	3
1.3.1	Bidomain Model	3
1.3.2	Monodomain Model	4
1.3.3	Nektar++: Cardiac EP Solver	5
1.3.4	Cell Models	5
1.4	Graph Neural Networks	6
1.4.1	Graph Representation	6
1.4.2	Multi-Layer Perceptrons (MLPs)	7
1.4.3	Graph Network (GN) Block	7
1.4.4	Multi-Scale GNN (MuS-GNN)	8
1.5	Rationale and Scope (Lit Review of Similar???)	9
2	Methodology	10
2.1	Simulation Data Generation: Nektar++ CardiacEPSolver	11
2.1.1	CardiacEPSolver Solver Info	11
2.1.2	CardiacEPSolver Parameters	11
2.1.3	CardiacEPSolver Stimuli	12
2.1.4	HP Refinement Study	13
2.2	Constructing a Graph Representation of the Data	13
2.3	Designing the GNN	14
2.3.1	Model Architecture	14
2.3.2	Training Procedure	14
2.3.3	Data Augmentation	15
2.4	Loss Function Analysis	15
2.4.1	Initial MuS Loss	15
2.4.2	Gradient Loss	16
2.4.3	Dissipation-Dispersion Loss Split	16
2.4.4	Gaussian Loss	17

2.4.5	Gradient Gaussian Combined Loss	17
2.4.6	Methodology for Comparing Loss Functions	17
2.5	Model Evaluation	18
2.5.1	Coefficient of Determination, R^2	18
2.5.2	Dissipation Dispersion Ratio	19
2.5.3	Wavefront and Edge Errors	19
2.5.4	Evaluation of Metrics	20
2.6	Measurement of Action Potential	20
3	Datasets	21
3.1	Datasets	21
3.1.1	Training Datasets	21
3.1.2	Testing Datasets	21
3.1.3	Limitations of our Datasets	23
4	Optimisation Results and Model Development	24
4.1	Computational Parameter Optimisation	24
4.1.1	HP Refinement Results	24
4.2	Network Architecture Selection	25
4.2.1	Hyperparameter Study Results	25
4.3	Comparing Loss Functions	26
4.3.1	Relative Loss Results	27
4.4	Integrated Model Configuration: AP-GNN	29
4.4.1	Training Performance	30
4.5	Integrated Model Configuration: AP-GNN-STIM	30
4.5.1	Training Performance	30
5	Model Performance: Results and Analysis	31
5.1	Network Performance: AP-GNN	31
5.2	AP-GNN-STIM	33
5.2.1	Stimuli Capture	34
5.2.2	AP Graph	34
5.3	Comparing to Nektar++ to solve	34
5.4	Testing on Non-Linear Domains	34
5.5	Limitations and Further Study	37
6	Conclusions	38
A		43
A.1	Bidomain Equation Derivation	43
A.2	Monodomain Equation Derivation	43
B		45
B.1	Mesh Production	45

B.2	<i>CardiacDataset</i> Class	45
C		46
C.1	Message Passing (MP) Algorithm	46

List of Figures

1.1	Diagram of the Cardiac Conduction System [5]	2
1.2	“Cardiac tissue architecture: (a) myocardium composed of myocytes; (b) individual myocyte with intercalated discs and gap junctions. [<empty citation>]	3
1.3	Node and attribute visualisation for a directed, attributed multi-graph with global attributes [22]	7
1.4	MLP Architecture	7
1.5	Multi-scale graph neural network (MuS-GNN) architecture [28].	8
2.1	The process through which data will be compiled to train and test the model	10
3.1	Snapshots of the transmembrane potential field, captured using Paraview [35], showing the morphology of our datasets.	22
4.1	Conduction velocity percentage error vs H and P.	24
4.2	Comparison of percentage errors for different parameters.	25
4.3	Runtime vs validation loss	26
4.4	Parallel coordinate plot showing hyperparameter combinations used during optimisation, with their validation loss (Objective Value)	26
4.5	Caption	27
4.6	Coefficient of Determination, R^2 , for the loss functions over the range of test scenarios. (Higher is better)	28
4.7	Loss function performance, measured using our Wavefront metric. (Lower is better)	28
4.8	Caption	29
4.9	Caption	29
5.1	Inference of AP-GNN on the Collision dataset. Ground truth, prediction and error	32
5.2	AP-GNN inference on long time dataset	32
5.3	Inference of action potential over 1.5 seconds by AP-GNN.	33
5.5	Action potential for AP-GNN-STIM.	34
5.4	AP-GNN-STIM predicting the evolution of a new stimulus, with the stimulus mask passed.	34
5.6	Distribution of edge lengths for the meshes, DS1, MedMesh and CoarseMesh	35
5.7	Performance of validation data.	35
5.8	Performance on a coarser mesh, MedMesh	36
5.9	Performance on a coarser mesh, CoarseMesh	36

List of Tables

1.1	Cell Models Available with Nektar++. [17]	6
3.1	Summary of datasets used for model training, validation, and testing.	22
5.1	Performance of the network evaluated on different datasets.	31
5.2	Performance of the network evaluated on different datasets.	33
5.3	Performance of the network evaluated on different datasets.	34

Abstract

Abstract goes here. **TODO:** Write abstract

Chapter 1

Introduction

1.1 Motivation and Aims

Transient dynamics in cardiac electrophysiology are clinically significant and computationally challenging phenomena in cardiovascular medicine [1]. Unlike steady-state behaviour, these transient events serve as precursors to life-threatening arrhythmias [2]. Current simulation methods require hours to days, limiting their clinical applicability for emergency interventions [3].

This work presents a novel approach to modelling these phenomena, utilising a graph neural network (GNN) framework developed by Mario Lino [4].

Specific Aims:

1. Produce training datasets using Nektar++ that will be used to train a model.
2. Develop single and multi-scale GNN architecture for cardiac transient prediction.
3. Produce a training procedure that optimises the model's ability to learn the cardiac EP phenomenology.
4. Demonstrate predictive capability that competes with traditional numerical methods at a clinically relevant time scale.

TODO: Add Action Potential Morphology

1.2 Cardiac Electrophysiology

The heart pumps blood around the body through precisely timed mechanical contraction of the cardiac tissue. This precise timing is achieved using a propagation of electrical signals through the heart tissue, triggering electrical activation of the muscles of the heart.

Electrical signals are initiated at the Sinoatrial node (pacemaker), and the tissue builds up charge to a threshold value. Once this threshold value is surpassed, voltage-gated ion channels

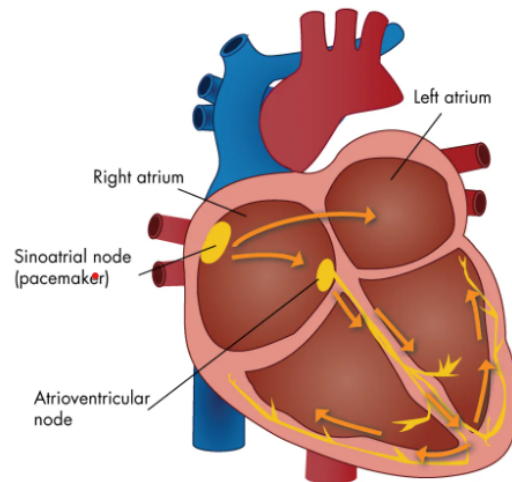


Figure 1.1: Diagram of the Cardiac Conduction System [5]

open, allowing for an influx of charged ions that depolarise the cell. The depolarisation generates a potential difference with neighbouring cells, triggering a chain reaction that propagates through the tissue. Once reaching the Atrioventricular node, the signal is delayed to allow time for fluid movement, after which the depolarisation continues down the intraventricular septum to the apex of the heart. At this point, the potential moves up through the ventricles to complete a full cycle of the pumping of the heart. This pathway can be seen in Figure 1.3.

Once depolarised, the ion channels close and the potential builds again to return to the polarised state.

1.2.1 The Microstructures of Cardiac Tissue

The myocardium, composed mainly of cardiac myocytes, is the bulk of the cardiac tissue, supporting electrical signal propagation. Myocytes contain the voltage-gated ion channels that open when the transmembrane voltage reaches a threshold, allowing an influx of ions that produce rapid depolarisation. Neighbouring myocytes are electrically coupled by low-resistance gap junctions composed of connexins [6], which provide pathways for local depolarisation of adjacent cells and enables anisotropic conduction across the tissue. In addition to myocytes, fibroblast cells are abundant, acting as connective tissue and can electrically couple to myocytes. They act as passive loads or current sources, modifying resting potential, action potential morphology, and conduction, particularly in fibrotic or remodelled myocardium. This cell-to-cell coupling underlies action potential propagation through the myocardium. [1].

1.2.2 The Macrostructures of Cardiac Tissue

Cardiac myocytes are linked end-to-end by gap junctions and organised into fibres that assemble into layers within the myocardium. Because electrical current flows most easily along the myocyte long axis, conduction is anisotropic: fastest along fibres, slower across the cell layer, and slowest perpendicular to the cell layers [7]. This layered architecture underlies the directional

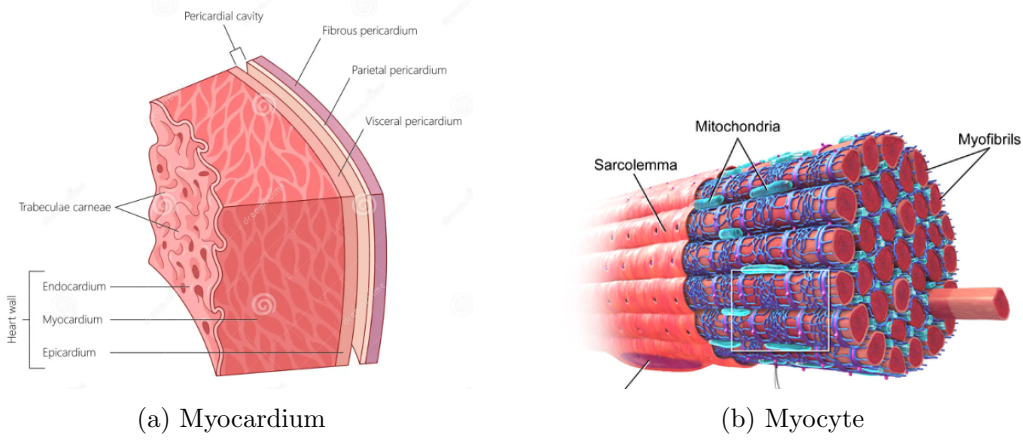


Figure 1.2: “Cardiac tissue architecture: (a) myocardium composed of myocytes; (b) individual myocyte with intercalated discs and gap junctions. [[empty citation](#)]

(anisotropic) propagation of the cardiac action potential [8].

1.2.3 Acquiring a Patient Specific Model of the Heart

While this work employs simplified geometries for proof-of-concept, real clinical deployment requires patient-specific models incorporating individual anatomical variations, scar tissue distribution, and medication effects [9, 10]. Magnetic resonance imaging (MRI) can be used to capture these patient specific characteristics, using methods outlined in the following work [11, 12, 13].

Current computational methods cannot achieve the speed necessary for patient-specific real-time predictions, creating a gap that our approach aims to address.

1.3 Mathematical Models of Cardiac Tissue

Mathematical models developed through the 20th century are used to model cardiac tissue. The models we will introduce work on the assumption that we can treat the tissue as a syncytium, a continuous medium, obtained by treating the discrete cell structure as a homogeneous anisotropic conductive medium.

1.3.1 Bidomain Model

The bidomain model of cardiac tissue, splits the heart tissue into intra- and extra-cellular domains. To these two domains, we apply Ohm’s current law and the laws of charge and current conservation [14].

Ohm’s current law is expressed in the form,

$$\mathbf{J} = -\mathbf{G}\nabla\phi \quad (1.1)$$

with J the current density, G the conductivity tensor and ϕ the electric potential.

We define the two domains, i, e , to be separated by a membrane. This allows us to define separate current transport equations for our intra- and extracellular domains within which Ohm's Law, (Equation 1.1) applies; this gives us,

$$\mathbf{J}_i = -\mathbf{G}_i \nabla \phi_i \quad (1.2)$$

$$\mathbf{J}_e = -\mathbf{G}_e \nabla \phi_e \quad (1.3)$$

The law of charge conservation dictates that no net flux of charge crosses the boundary, giving the following relationship between the domains.

$$\nabla \cdot \mathbf{J}_i = -\nabla \cdot \mathbf{J}_e = I_m \quad (1.4)$$

where I_m is the transmembrane current per unit volume with units $[Am^{-3}]$.

Electrically, the cell membrane behaves electrically as a capacitor in parallel with the ionic pathways. Therefore, the transmembrane current density per unit area I_m is the sum of a capacitive current, $C_m \frac{dV_m}{dt}$ and an ionic current $i_{ion}(V_m, w)$, where w denotes ionic gating variables [15]. Homogenising to per-unit-volume gives,

$$I_m = \beta_m \left(C_m \frac{dV_m}{dt} + i_{ion} \right) \quad (1.5)$$

where β_m is the surface area to volume ratio of the cardiac wall, C_m is the capacitance of the connective membrane, and V_m is a transmembrane voltage which is related to the relative potentials by $V_m = \phi_i - \phi_e$.

Combining Equations (1.2)-(1.5), as seen in Appendix A.1, yields our governing equations,

$$\begin{aligned} \nabla \cdot \mathbf{G}_i (\nabla V_m + \nabla \phi_e) &= \beta_m \left(C_m \frac{dV_m}{dt} + i_{ion} \right) \\ \nabla \cdot ((\mathbf{G}_i + \mathbf{G}_e) \nabla \phi_e) &= -\nabla \cdot (\mathbf{G}_i \nabla V_m) \end{aligned}$$

Further imposing the assumption of an isolated myocardium with no net current crossing the outer surface ∂H , homogeneous Neumann conditions apply at the boundary, giving,

$$\hat{\mathbf{n}} \cdot (\mathbf{G}_i \nabla (V_m + \phi_e)) = 0 \quad (1.6)$$

$$\hat{\mathbf{n}} \cdot (\mathbf{G}_e \nabla \phi_e) = 0 \quad (1.7)$$

with respect to the unit normal, $\hat{\mathbf{n}}$.

1.3.2 Monodomain Model

For the Monodomain model, we make the assumption that the anisotropies of both the intra- and extracellular regions are equivalent and therefore the conductivities of the two regions are

proportional. This can be expressed as,

$$\mathbf{G}_e = \lambda \mathbf{G}_i \quad (1.8)$$

where $\lambda > 0$ is some constant proportionality.

Eliminating the extracellular potential from the bidomain equations then yields a single reaction–diffusion equation for the transmembrane potential $V_m = \phi_i - \phi_e$, from 1.3.1 into our Monodomain governing equation,

$$\nabla \cdot \mathbf{G} \nabla V_m = \beta_m \left(C_m \frac{dV_m}{dt} + i_{ion} \right) \quad (1.9)$$

with effective conductivity,

$$\mathbf{G} = \frac{\lambda}{\lambda + 1} \mathbf{G}_i \quad (1.10)$$

For an isolated tissue, the reformulated homogeneous Neumann boundary conditions apply,

$$\hat{\mathbf{n}} \cdot \mathbf{G} \nabla V_m = 0 \quad (1.11)$$

This derivation can be seen in Appendix A, and it follows on that (1.9) can be reformulated into explicit time form as a function of the transmembrane potential as follows.

$$\frac{\partial V_m}{\partial t} = \nabla \cdot (\mathbf{D} \nabla V_m) - \frac{i_{ion}}{C_m} \quad (1.12)$$

defining the diffusion tensor $\mathbf{D} = \mathbf{G}/(\beta_m C_m)$.

Under the proportional anisotropy assumption, monodomain and bidomain produce very similar activation patterns and electrograms for intrinsic activation without external stimulation, with reported differences in conduction speed on the order of a few percent in whole-heart simulations [16].

1.3.3 Nektar++: Cardiac EP Solver

We employ Nektar++ for solving the Monodomain equation (??). This is a spectral/hp element framework for high-accuracy data generation that includes a numerical solver, CardiacEPSolver. The numerical framework of high-order polynomial representations allows efficient capture of steep action potential wavefronts across complex geometries. Details of the numerical implementation can be found in the documentation [17].

1.3.4 Cell Models

For the cardiac governing equation, we need to employ a cell model to determine the ionic charges, i_{ion} . The Nektar++ CardiacEP solver supports a number of these listed in Table 1.1, each with their own advantages and disadvantages.

For the purpose of our simulations, the Courtemanche cell model will be used [18]. The

Courtemanche model has 20 parameters, which model the major ionic currents, pumps and exchangers. Firstly, this promotes improved accuracy of the model with respect to human heart tissue due to the biophysical parameters on which the model relies. Secondly, the extra fields modelled by the Courtemanche model will also allow extra information to be passed to the network during the training phase, in theory enabling the model to better characterise the human cardiac tissue. However, this model will increase simulation times due to the added complexity; to remove this issue, a model such as the Aliev-Panfilov, a 2-parameter model, could be used for rapid data production [19].

Model	Description	No. of Var.	Ref.
AlievPanfilov	Phenomological	1	[19]
CourtemancheRamirezNattel98	Human atrial	20	[18]
FitzHughNagumo			
Fox02			
LuoRudy91	Mammalian ventricular	7	[20]
PanditGilesDemir03			
TenTusscher06	Human ventricular	18	[21]
Winslow99			

Table 1.1: Cell Models Available with Nektar++. [17]

TODO: Make this section flow more naturally

1.4 Graph Neural Networks

1.4.1 Graph Representation

From now on, we will use 'graph' to mean a "directed, attributed multi-graph with a global attribute" as described by Battaglia et al. [22].

Specifically, as in their notation, our graph data structure has the form $G = (\mathbf{u}, V, E)$, where $\mathbf{u}$ is our global attribute, $V = \{\mathbf{v}_i\}_{i=1:N^v}$ the set of node attributes attached to each node, and $E = \{(\mathbf{e}_k, r_k, s_k)\}_{k=1:N^e}$ the set of edges, each described by the coordinates of the send and receive nodes to which the said edge corresponds.

TODO: FIND BETTER IMAGE

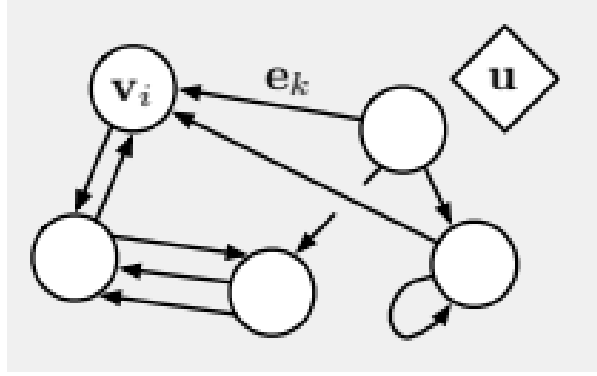


Figure 1.3: Node and attribute visualisation for a directed, attributed multi-graph with global attributes [22]

1.4.2 Multi-Layer Perceptrons (MLPs)

A neural network (NN) is a supervised learning algorithm built from layers of units that apply weighted sums and nonlinear activations, mapping inputs to outputs, allowing for the approximation of functions. Information flows from an input layer, through a specified number of hidden layers, and into the output in a manor that minimises a loss function. [23, 24, 25]

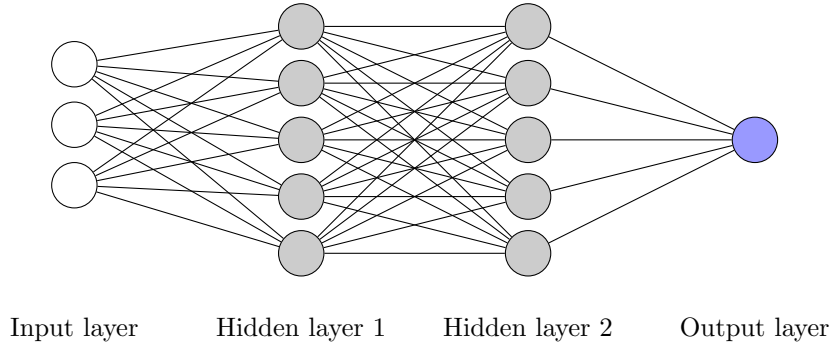


Figure 1.4: MLP Architecture

A multi-layer perceptron (MLP) is a feed-forward neural network (NN) architecture, typically with fully connected layers and nonlinear activations, that is capable of representing highly nonlinearly separable functions. [26]

1.4.3 Graph Network (GN) Block

The logic of the GN block, which updates edge, node, and global attributes in three sequential stages, follows the formalism of Battaglia et al. (2018)[22]. This can be found in Appendix C, a brief overview of the logic is as follows.

The GN block is the logic that allows us to define relationships between nodes. Each GN block contains two MLPs, one that transfers the attributes from the sender node to the edges, and a second that aggregates the contribution from all edges culminating at a node, updating the node attributes.

For each individual update of the GN block, the algorithm first updates all the edge attributes. Then, through a process of aggregation of edges terminating at the given node, the node attributes are updated. Then finally the global attributes of the graph are updated through an aggregation of all edge and node attributes.

This manifests as a GN block that must perform three update functions (denoted as ϕ) and three aggregation functions (denoted as ρ). These can be written as [22]:

$$e'_k = \phi^e(e_k, v_{r_k}, v_{s_k}, u) \quad (1.13)$$

$$v'_i = \phi^v(\bar{e}'_i, v_i, u) \quad (1.14)$$

$$u' = \phi^u(\bar{e}', \bar{v}', u) \quad (1.15)$$

$$\bar{e}'_i = \rho^{e \rightarrow v}(E'_i) \quad (1.16)$$

$$\bar{e}' = \rho^{e \rightarrow u}(E') \quad (1.17)$$

$$\bar{v}' = \rho^{v \rightarrow u}(V') \quad (1.18)$$

This process allows for the encoding of complex dynamics and the passing of messages or signals between adjacent nodes. This architecture mimics, to some extent, the physical nature of the reaction diffusion equation that governs the action potential propagation. Therefore, there is an assumption that for a suitable MP function, the dynamics of Cardiac Electrophysiology could be captured.

The aggregation operators ρ (e.g., sum or mean) ensure permutation invariance and stable message passing, while directed, attributed edges allow encoding of domain-specific structure such as anisotropy and geometric distances

1.4.4 Multi-Scale GNN (MuS-GNN)

The network architecture that we use has been built on the PyTorch platform [27], by Mario Lino [28].

The network can construct lower-resolution representations of the graph, $H^l = (V^l, E^l, A^l)$ for $l = 2, \dots, L$. The set of nodes at each lower-order representation is a subset of the nodes from the higher-order representations, $V^l \subset V^{l-1}$. This subset of nodes is obtained using Guillard's coarsening algorithm [29]. Then, on this lower order representation of the domain, the same MP logic is applied, allowing for messages to be passed over greater distances.

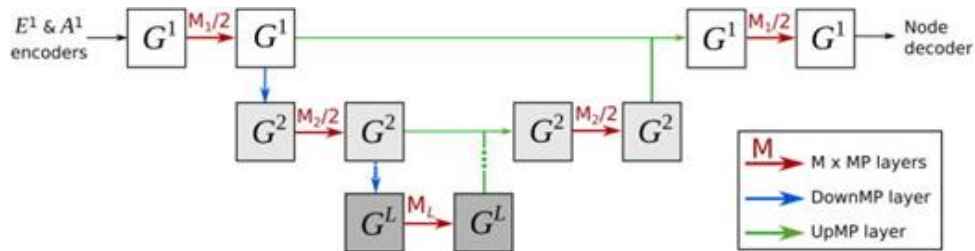


Figure 1.5: Multi-scale graph neural network (MuS-GNN) architecture [28].

1.5 Rationale and Scope (Lit Review of Similar???)

Choices based on other studies

[30]

[31]

[<empty citation>]

Chapter 2

Methodology

The first step towards producing a capable MuS-GNN framework that can predict the transient dynamics of the action potential in cardiac tissue is to generate high-fidelity, physically relevant simulations that will be used as training datasets. Then, we will convert our simulations to graph representations to train our model, with the intention of accurately predicting conduction velocity and repolarisation rate. We will introduce two network architectures: AP-GNN, designed to capture the evolution of the cardiac waves and AP-GNN-STIM, designed to further capture the production of the potentials. If the network is learning the underlying physical phenomena, then it should be able to generalise and capture more complex dynamics. These would include wave collisions and later spiral waves, a model capable of this will have physical and potential clinical relevance.

The following chapter provides an overview of the methods used to produce training data, set up an efficient network architecture and training procedure, and then eventual evaluation of the models produced.

TODO: Clean up or remove

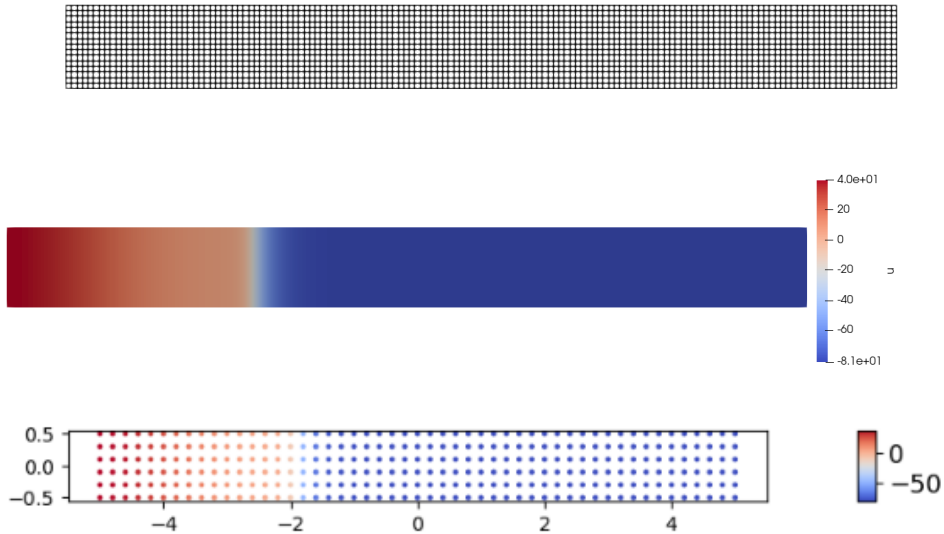


Figure 2.1: The process through which data will be compiled to train and test the model

2.1 Simulation Data Generation: Nektar++ CardiacEPSolver

All data production will be done using the Nektar++ CardiacEPSolver. We need to decide on the solver parameters that will be used to produce the datasets and set up a configuration file to pass to Nektar++, which will define the domain, boundary conditions, solver options and cell model parameters.

We then produce a mesh to simulate on using the meshing software, Gmsh, "an open source 3D finite element mesh generator" [32].

TODO: Potentially move to appendices and summarise

2.1.1 CardiacEPSolver Solver Info

The first thing we will set up is the solver info, which determines the set of equations that will be solved and what methods are used. As seen in Listing 2.1, we will be modelling the transmembrane potential using the monodomain equation 1.9. This assumes the anisotropies of the intra- and extracellular regions are equivalent and continuous. This will then be paired with the Courtemanche-Ramirez-Nattel human atrial cell model [18]. This will preserve the biophysical realism in the simulations, as well as allowing for the use of additional state variables and currents to train the model if required.

Listing 2.1: CardiacEPSolver: Solver Info

```

1 <SOLVERINFO>
2   <I PROPERTY="EQTYPE" VALUE="Monodomain" />
3   <I PROPERTY="CellModel" VALUE="CourtemancheRamirezNattel98" />
4   <I PROPERTY="CellModelVariant" VALUE="Original" />
5   <I PROPERTY="Projection" VALUE="Continuous"/>
6   <I PROPERTY="DiffusionAdvancement" VALUE="Implicit"/>
7   <I PROPERTY="TimeIntegrationMethod" VALUE="IMEXdirk_3_4_3"/>
8 </SOLVERINFO>

```

2.1.2 CardiacEPSolver Parameters

TODO: Provide proper motivation/

Next, looking at the methods used to solve our cardiac PDEs, we implement a continuous projection at element boundaries, which promotes numerical stability in diffusion problems [<empty citation>]. We have an implicit time-stepping for the diffusion advancement, these are unconditionally stable for diffusion problems as opposed to explicit time-stepping [<empty citation>]. Then the time integration is handled using an 'IMEX' scheme, according to the Nektar++ documentation, "these schemes support the concept of being able to split right hand forcing term into an explicit and implicit component [17].

We, define the parameters to be used by our solver as follows. The time stepping, $dt = 0.02$ ms, is chosen to promote numerical stability, and then consequently the cell model has 4 substeps,

as required by the Courtemanche cell model, $dt < 0.005$ ms for stability [18]. The other time or step-related variables determine the length of our simulation and when we take checkpoint measurements. Then the final parameters to mention are, χ, C_m and o_{max} , all physical tissue parameters chosen to be physically relevant [33].

Listing 2.2: CardiacEPSolver: Parameters

```

1 <PARAMETERS>
2   <P> IO_TimeStep      = 0.2   </P> <!-- time step for output -->
3   <P> IO_NumSteps      = 50    </P> <!-- number of output steps -->
4   <P> TimeStep         = 0.02  </P>
5   <P> FinTime          = IO_TimeStep * IO_NumSteps </P>
6   <P> NumSteps         = FinTime/TimeStep </P>
7   <P> IO_CheckSteps    = 1/TimeStep </P>
8   <P> IO_InfoSteps     = IO_CheckSteps </P>
9
10  <P> IterativeSolverTolerance = 1e-05 </P>
11  <P> Chi = 140 </P> <!-- larger: wavefront moves slower -->
12  <P> Cm = 0.01 </P> <!-- smaller: higher peak mag. of act. -->
13  <P> Substeps = 4 </P> <!-- Courtemanche needs about dt=0.005 -->
14  <P> o_max = 0.13341 </P> <!-- Effective conductivity of gap
    junctions -->
15 </PARAMETERS>

```

2.1.3 CardiacEPSolver Stimuli

For the production of training and testing data a range of stimuli will need to be produced on various domains to better understand where our models succeed and with what they fail.

Nektar++ can produce circular and rectangular stimuli affecting the heart tissue through a simulated injection of charge. We define the geometric position of the stimuli within the domain, the charge per unit area (p_strength) and the spread of said charge (p_is). These values are chosen to provide a stimulus that triggers the action potential wave. An example of can be seen in Listing 2.3.

Listing 2.3: CardiacEPSolver: Stimuli

```

1 <STIMULI>
2   <STIMULUS ID="0" TYPE = "StimulusRect">
3     <p_x1> -5.0</p_x1>
4     <p_y1> -2.5</p_y1>
5     <p_z1> 0.0</p_z1>
6     <p_x2> -4.5</p_x2>
7     <p_y2> 2.5</p_y2>
8     <p_z2> 0.0</p_z2>
9     <p_is> 100.0</p_is>
10    <p_strength> 20.0 </p_strength>
11
12    <PROTOCOL TYPE = "ProtocolSingle">

```

```

13         <START> 0.0 </START>
14         <DURATION> 2.0 </DURATION>
15     </PROTOCOL>
16 </STIMULUS>
17 </STIMULI>

```

Having established the solver configuration, cell model parameters, and stimulation protocols, we need to choose a numerical discretisation that adequately captures the key aspects of the action potential morphology. The accuracy of our simulations, and consequently the quality of our machine learning training data, depends heavily on the spatial resolution and polynomial order used in the spectral/hp element method. Therefore, complete a systematic mesh and polynomial order (HP) refinement to achieve the optimal balance between computational efficiency and numerical accuracy.

2.1.4 HP Refinement Study

Cardiac electrophysiology features steep action potential wavefronts that can induce Gibbs oscillations and dispersion when under-resolved with high-order methods [34]. To select a cost-effective discretisation that preserves conduction velocity and action potential morphology, a combined h- and p-refinement study was performed.

Firstly, h-refinement reduces element size at fixed polynomial order and is effective at resolving sharp wavefronts. Then, p-refinement increases polynomial order at a fixed mesh and attains high accuracy in smooth regions, but is more prone to oscillations at sharp transitions. Both strategies were evaluated to determine the minimal resolution that preserves activation timing fidelity

A high-fidelity reference was generated using polynomial order $P=13$ and 20 nodes/mm (element size 0.05 mm). Candidate discretisations spanning coarser element sizes and lower P were benchmarked against this reference. Accuracy was quantified via conduction velocity (CV) error, computed from the wavefront isochrone at $V_m = 0$ mV over a fixed time window using standard measurement tools in ParaView [35]. The CV percentage error provided a single decision metric for selecting the lowest-cost configuration that maintains acceptable propagation accuracy for training data generation.

TODO: CAN ADD VISUALISATION IF NEEDED

2.2 Constructing a Graph Representation of the Data

To convert our simulations to graph representations, we employ the following data pipeline, adapted from the methods outlined by Mario Lino [36].

Mesh-to-Graph Conversion

Nektar++ finite element meshes are converted to graph representations where:

- **Nodes:** Represent mesh vertices with attributes including spatial coordinates, transmembrane potential, and ionic variables
- **Edges:** Bidirectional, mesh adjacent elements, with the attribute of length
- **Global attributes:** Encode simulation parameters and boundary conditions

All codes and implementations for this can be found in Appendix B **TODO: Potentially move to appendices/ reference appendix where codes are**

2.3 Designing the GNN

The implementation of our graph neural network architecture uses the structures developed by Mario Lino [28]. We have added code that allows us to pass our cardiac simulations to train effectively on the MuS-GNN architecture provided.

2.3.1 Model Architecture

This architecture encodes domain structure through node and edge features. The network then stacks several message-passing blocks, each using MLPs for edge and node updates and a mean as the permutation-invariant aggregation function. For our experiments, a single-scale message-passing stack is used rather than multi-scale coarsening. This decision is based on the assumption that local reaction–diffusion dynamics are captured effectively locally without the need for larger-scale information passing. However, this assumption should be challenged as there exist larger scale dynamics, such as repolarisation or wavefront morphology, that could be better captured with the inclusion of coarsened graphs.

TODO: Talk about Multi v single when have final results

Due to the reaction diffusion phenomenology of the action potential being a local phenomena, we hypothesised that no extra information would be gleaned from having a multi-scale approach to the problem.

2.3.2 Training Procedure

To train the network, we take an instance of the graph and propagate said instance through our network with the goal of predicting the next timestep. From which we can compute a loss, how we calculate this loss is investigated thoroughly, which will allow us to backpropagate to update the network’s parameters. We will employ hyperparameter tuning to optimise training

Bayesian Optimisation (Hyperparameter Tuning)

To process the large hyperparameter space generated by the model we tuned model parameters by performing a Bayesian optimisation [37]. This process employs a Tree-structured Parzen

Estimator [38] which performs an automated search to select key settings using validation performance as the target. Early pruning and seed averaging improve efficiency and stability.

2.3.3 Data Augmentation

We apply a number of transformations to data passed to the network; these, one intend to improve generalisation, and two allow us to set up data in a way the network can learn from. These transformations include geometric transforms; rotations to mitigate a directional bias,... We also add additional noise to the data.

2.4 Loss Function Analysis

TODO: Clean up explanations / make shorter

The selection of an appropriate loss function dictates a models ability to learn the desired dynamics; a good function will allow for accurate representation at the wave front, correctly predict the spatio-temporal evolution, and capture the repolarisation rate. We require a loss function that acts with computational efficiency while capturing the underlying biophysical phenomena.

Research has shown that different loss functions can lead to substantially different model behaviour, even when applied to identical architectures and datasets [**<empty citation>**]. For a model tasked with learning cardiac electrophysiology, traditional loss functions may focus loss reduction efforts on non-critical features. Furthermore, the multi-scale nature of cardiac electrophysiology, spanning from cellular-level dynamics to tissue-level propagation, necessitates loss functions that can effectively balance these competing scales during optimisation.

What each one hopes to achieve and a measurement of whether they were considered successful or not.

Table: metrics describing the success

2.4.1 Initial MuS Loss

The foundational loss function for this investigation is from the multi-scale (MuS) framework developed by Lino et al. [28], designed to capture the dynamics in Eulerian fluid flow. This MuS loss function combines domain-wide accuracy and boundary-specific constraints through a hybrid MSE-L1 function as follows,

$$L(\theta) = \sum_{i \in V^1} (\mathbf{u} - \mathbf{u}_{gt})^2 + \lambda_d \sum_{i \in \partial V^1} |\mathbf{u} - \mathbf{u}_{gt}| \quad (2.1)$$

2.4.2 Gradient Loss

The phenomonolgy of cardiac action potential propagation is characterised by steep wavefronts at the leading edge of excitation waves. This sharp gradient is a fundamental property of conduction in the myocardium and is critical for accurate timing. leads us to want to enforce this steep (larger) gradient. It has been observed that traditional MSE-based loss functions can struggle to preserve the steep nature of the wavefronts, often leading to numerical diffusion that compromises the physiological identity of the wavefront [**<empty citation>**].

The Gradient Loss function specifically targets the preservation of steep wave front gradients by incorporating an explicit spatial derivative term into the optimisation objective. This is incorporated as follows,

$$L(\theta) = \sum_{i \in V^1} (\mathbf{u} - \mathbf{u}_{gt})^2 + \lambda_d \sum_{i \in \partial V^1} |\mathbf{u} - \mathbf{u}_{gt}| + \lambda_g \sum_{i \in V^1} (\nabla \mathbf{u} - \nabla \mathbf{u}_{gt})^2 \quad (2.2)$$

In practice, this was implemented by taking a quasi-first-order estimation of the derivatives,

$$\nabla \mathbf{u}_i \approx \sum_k (\mathbf{u}_i - \mathbf{u}_k) / |\mathbf{x}_i - \mathbf{x}_k| \quad (2.3)$$

where k are the nearest neighbours.

2.4.3 Dissipation-Dispersion Loss Split

Then we will also experiment with splitting the MSE loss into quasi-dispersive and dissipative terms as in Deo [39]. The dispersive component primarily affects wave timing and phase relationships, while the dissipative component measures errors in amplitude and energy conservation. This split allows us to explicitly weight the importance of accurate wavefront timing, while tolerating small errors in wave amplitude, which are less physiologically critical given our lower-order simulations.

Let $\mathbf{u}_a$ denote the predicted values and $\mathbf{u}_{gt}$ the ground truth values.

$$\tau_{\text{DISS}} = \left[\sqrt{\frac{1}{N} \sum_{i \in V^1} (u_i - \bar{u})^2} - \sqrt{\frac{1}{N} \sum_{i \in V^1} (u_{gt,i} - \bar{u}_{gt})^2} \right]^2 + (\bar{u} - \bar{u}_{gt})^2, \quad (2.4)$$

$$\tau_{\text{DISP}} = 2(1 - \rho) \sqrt{\frac{1}{N} \sum_{i \in V^1} (u_i - \bar{u})^2} \sqrt{\frac{1}{N} \sum_{i \in V^1} (u_{gt,i} - \bar{u}_{gt})^2}, \quad (2.5)$$

$$L(\theta) = \lambda_{\text{diss}} \tau_{\text{DISS}} + \lambda_{\text{disp}} \tau_{\text{DISP}}, \quad (2.6)$$

where

$$\rho = \frac{\sum_{i \in V^1} (u_i - \bar{u})(u_{gt,i} - \bar{u}_{gt})}{\sqrt{\sum_{i \in V^1} (u_i - \bar{u})^2} \sqrt{\sum_{i \in V^1} (u_{gt,i} - \bar{u}_{gt})^2}}. \quad (2.7)$$

$$\tau_{\text{DISS}} = [\sigma(u) - \sigma(u_{gt})]^2 + (\bar{u} - \bar{u}_{gt})^2, \quad (2.8)$$

$$\tau_{\text{DISP}} = 2(1 - \rho)\sigma(u)\sigma(u_{gt}), \quad (2.9)$$

$$L(\theta) = \lambda_{\text{diss}}\tau_{\text{DISS}} + \lambda_{\text{disp}}\tau_{\text{DISP}}. \quad (2.10)$$

Here, σ is the standard deviation, $\bar{u}$ is the mean, and λ_{diss} , λ_{disp} are tunable hyperparameters that balance the relative contributions of dissipation and dispersion errors. *When two wave patterns differ only in amplitude but have identical phase, $\rho = 1$ and $\tau_{\text{DISP}} = 0$.*

2.4.4 Gaussian Loss

The Gaussian loss aims to specifically emphasise the critical phases of the cardiac action potential propagation. Namely, the rapid depolarisation where transmembrane potentials transition from approximately -85mV to +20mV. This focus will allow the model to focus on the most physiologically relevant portions of the action potential. This is achieved using a Gaussian weighting centred about a transmembrane potential of -35mV, such a potential occurs during depolarisation spike as well as late in the repolarisation phase, allowing the model to focus on two key phenomena.

This loss is implemented by applying a Gaussian mask across the mesh, multiplying all values by some Gaussian curve centred at -35mV with a s.d. of +-15mV.

$$L(\theta) = \text{MSE}(u - u_{GT}) + \text{MSE}(u_{\text{Gaussian}} - u_{\text{Guassian}(GT)}) \quad (2.11)$$

2.4.5 Gradient Gaussian Combined Loss

We will also test a loss function that combines the gradient and gaussing losses as follows

$$L(\theta) = \sum_{i \in V^1} (\mathbf{u} - \mathbf{u}_{gt})^2 + \lambda_d \sum_{i \in \partial V^1} |\mathbf{u} - \mathbf{u}_{gt}| + \lambda_g \sum_{i \in V^1} (\nabla \mathbf{u} - \nabla \mathbf{u}_{gt})^2 + \text{MSE}(u_{\text{Gaussian}} - u_{\text{Guassian}(GT)}) \quad (2.12)$$

2.4.6 Methodology for Comparing Loss Functions

The comparative analysis of loss functions requires fair comparison across all approaches, with each loss being evaluated with identical network architecture, training procedures and computational resources to isolate the implications of each loss function.

Each loss function has gone through a process of hyperparameter tuning for its specific regularisation parameters. We will train the models on our first dataset with the same preprocessing and augmentation strategies applied.

The comprehensive evaluation will employ multiple metrics to assess the performance at

capturing the relevant phenomena.

We will use for primary accuracy metrics, the coefficient of determination (R^2) that measures the overall correlation between the prediction and ground truth as well as the mean squared error (MSE). Then, to quantify some cardiac-specific properties that we want our model to capture, we will observe the quasi-dissipative/dispersive split of error, analogous to an amplitude/phase error split. On top of this will observe the action potential morphology, looking for the preservation of the waveform.

The evaluation of these models will take place over four datasets, each designed to evaluate a different part of the morphology; these are the training set (action potential propagation), a planar wave (generalisation to different wave fronts), a collision of cardiac waves (unseen phenomena) and a long time large domain instance to capture the repolarisation.

2.5 Model Evaluation

TODO: Clean up make shorter?

This section outlines the framework used to evaluate model performance. We employ four complementary numerical metrics alongside qualitative visual inspection of the model outputs. The numerical metrics quantify overall predictive accuracy, dissipation behaviour, wavefront fidelity, and boundary performance, while visualisation helps capture patterns that are difficult to summarise via a single statistic.

2.5.1 Coefficient of Determination, R^2

The first metric considered is the coefficient of determination (R^2), which measures the proportion of variance in the target variable that is captured by the model predictions [**<empty citation>**]. Values close to 1 indicate near-perfect agreement, while values near 0 suggest no correlation between predicted and observed outcomes.

In our implementation, nodes located at domain boundaries are excluded from the computation. This adjustment reflects the fact that boundary artefacts often arise from numerical effects rather than physical dynamics, and including them would bias the evaluation. Formally:

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = 1 - \frac{\sum_{i \in M} (y_i - \hat{y}_i)^2}{\sum_{i \in M} (y_i - \bar{y})^2} \quad (2.13)$$

where SS_{res} is the residual sum of squares, SS_{tot} the total sum of squares, $\bar{y}$ the mean target value, and M the set of non-boundary nodes. Masking is included to maintain numerical stability, particularly when $\bar{y}$ approaches y_i

TODO: Fix maths, make consistent/ more easily readable don't need all the detail

$$SS_{res} = \sum_{i \in M} (y_i - \hat{y}_i)^2 \quad (\text{Residual Sum of Squares}) \quad (2.14)$$

$$SS_{tot} = \sum_{i \in M} (y_i - \bar{y})^2 \quad (\text{Total Sum of Squares}) \quad (2.15)$$

$$\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i \quad (\text{Mean of target values}) \quad (2.16)$$

$$M = \{i : y_i \neq \bar{y}\} \quad (\text{Mask to avoid division by zero}) \quad (2.17)$$

2.5.2 Dissipation Dispersion Ratio

The dissipation-dispersion ratio, derived from the work of [39], provides a measure of how the model balances the preservation of both energy dissipation and wave dispersion during propagation.

2.5.3 Wavefront and Edge Errors

Errors near the wavefront and at domain boundaries are of particular importance, as they strongly influence conduction properties and the accuracy of propagation dynamics.

We will also define metrics to describe both the error at the wavefront and the error at the boundary.

Boundary Errors

To assess boundary behaviour, we restrict attention to nodes along the outer edges of the computational domain. An L_1 norm of prediction error is then applied:

$$\mathcal{L}_{boundary} = \frac{1}{|B|} \sum_{i \in B} |y_i - \hat{y}_i| \quad (2.18)$$

where B denotes the set of boundary nodes.

Wavefront Errors

To isolate nodes lying on the propagating wavefront, we apply a Gaussian weighting mask designed to highlight regions where transmembrane potential lies near the depolarisation threshold. Specifically:

$$w_i = \exp\left(-\frac{(\hat{y}_i + 35)^2}{2\sigma^2}\right), \quad \sigma = 20 \quad (2.19)$$

This mask emphasises values around -35 mV, this is chosen as a good value to effectively pick out the wavefront as a value in excess of the threshold. The weighted error is then:

$$\mathcal{L}_{wavefront} = \frac{1}{|D|} \sum_{i \in D} w_i \cdot |y_i - \hat{y}_i| \quad (2.20)$$

2.5.4 Evaluation of Metrics

The combination of these metrics provides a holistic assessment of model quality. The R^2 score captures general predictive agreement, while the dissipation-dispersion ratio evaluates dynamical fidelity. Boundary and wavefront error metrics focus on localised phenomena that are particularly important for accurate electrophysiological modelling.

In addition to these quantitative measures, qualitative visualisations of model predictions are examined. These visual assessments highlight spatiotemporal features such as wave curvature, conduction block, or spiral tip dynamics, which may not be captured by summary statistics. Together, these complementary approaches provide a robust framework for comparing models, identifying failure modes, and understanding trade-offs between accuracy and efficiency.

Together, these metrics provide a comprehensive evaluation of model quality: R^2 captures overall accuracy, the dissipation-dispersion ratio reflects dynamical fidelity, and boundary/wavefront errors assess key local phenomena. Complementary visualisations further reveal spatiotemporal features not captured by statistics, enabling robust comparison of models and their trade-offs.

2.6 Measurement of Action Potential

We want to measure how our models handle inference on the scale of seconds, the same scale as whole heart operation. To do this, the model needs to be able to capture the rate of repolarisation. This will be measured using the AP domain, Figure 3.1f, and we will measure how the transmembrane potential evolves at the opposite end of the domain to the stimulus, allowing us to capture differences in conduction velocity and in repolarisation rate.

Chapter 3

Datasets

3.1 Datasets

Next, we provide an overview of the datasets we have produced. For the training datasets, we describe the motivations and outcomes we hope to achieve. And for testing datasets, each will be used to test a specific .

3.1.1 Training Datasets

The first dataset is designed to train our initial model to capture the basic behaviour of action potential propagation. We apply a single circular stimulus at a point in a 10×10 mm domain and allow it to propagate for 2 ms before recording. This ensures the stimulus is fully formed without capturing the details of the stimulus's evolution. Data is then recorded from $t = 2$ to $t = 12$ ms, producing 50 time steps per run (recorded every $\Delta t = 0.2$ ms), with 100 runs in total Figure 3.1a. Only the transmembrane potential, V_m , is recorded in this dataset.

The second dataset is intended to train a model to capture the stimulus itself with knowledge of when and where the stimuli occur, as well as better capture the repolarisation phase of the action potential. We increase the domain size to 20×20 mm and extend the simulation time to 20 ms. Two stimuli are applied: one at $t = 0$ ms and another at a random time within $[20, 40]$ ms. This setup allows us to observe how the tissue responds to multiple activations, shown in Figure 3.1b. Unlike the first dataset, here we record all 18 state variables from the Courtemanche cell model in addition to the transmembrane potential and a binary mask indicating where and when a stimulus is applied.

3.1.2 Testing Datasets

To test model performance, we construct a set of targeted scenarios with different geometries and stimulation protocols. Each dataset isolates a different challenge for the model:

Rectangular: A 10×10 mm domain with a single stimulus at $t = 0$. Used to test whether the model can reproduce simple planar wave propagation.

Collision: Two simultaneous stimuli initiated in a 10×10 mm domain. This tests the model’s ability to represent wave collisions.

Long Time: A larger 50×50 mm domain with a single stimulus at $t = 0$, recorded for a longer duration. This scenario evaluates wave stability and whether the model can maintain accurate dynamics over extended timescales.

AP: A narrow 1×10 mm domain with repeated stimuli applied every 1500 ms, producing a long sequence of action potentials. This dataset assesses the model’s ability to capture the long-term temporal behaviour and multiple AP cycles.

Table 3.1: Summary of datasets used for model training, validation, and testing.

Dataset	Purpose	D (mm)	Stimuli (ms)	Fields Recorded	Runs / Frames
DS1	Training	10×10	1 at $t = 0$	V_m only	100 / 50
DS2	Training	20×20	2: $t = 0, [20, 40]$	Courtemanche+Stim	100 / 100
Rect	Testing	10×10	1 at $t = 0$	V_m only	1 / 50
Collision	Testing	10×10	2 at $t = 0$	V_m only	20 / 100
LongTime	Testing	50×50	1 at $t = 0$	V_m only	10 / 150
AP	Testing	1×10	5: every 1500	Courtemanche+Stim	1 / 7500

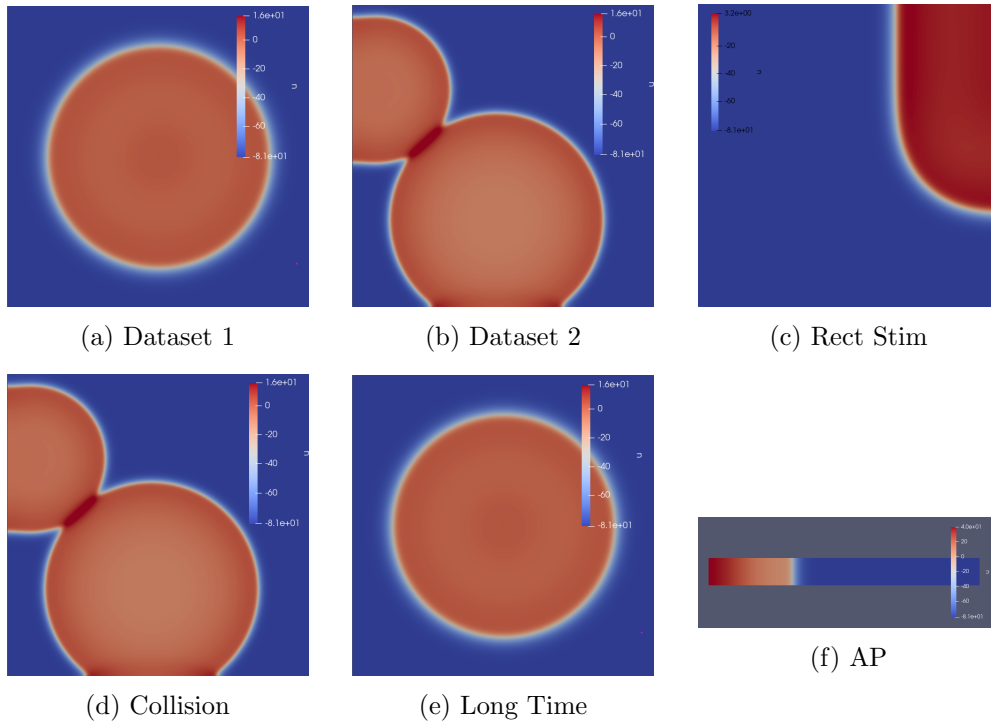


Figure 3.1: Snapshots of the transmembrane potential field, captured using Paraview [35], showing the morphology of our datasets.

3.1.3 Limitations of our Datasets

One limitation of the Nektar++ simulations is the use of Neumann boundary conditions. These enforce a constant flux at the edges of the domain, which can introduce boundary effects that may not be physiologically realistic. To avoid the model learning these potentially unphysical interactions, we may later exclude boundary regions from the dataset.

TODO: Make tables look nice and produce DS d e images

TODO: Fit to 2 pages

Chapter 4

Optimisation Results and Model Development

4.1 Computational Parameter Optimisation

The accurate modelling of AP morphology, and specifically the conduction velocity, is highly dependent on the resolution of our simulation mesh. Small errors will dramatically alter wave dynamics over longer time frames. This section investigates the optimal balance between computational efficiency and accuracy through hp-refinement of our discontinuous Galerkin discretisation.

4.1.1 HP Refinement Results

Measuring the percentage error in the conduction velocity allows us to choose modelling parameters that will produce simulations with maximum accuracy at the wavefront and allow for the capture of the correct conduction velocity.

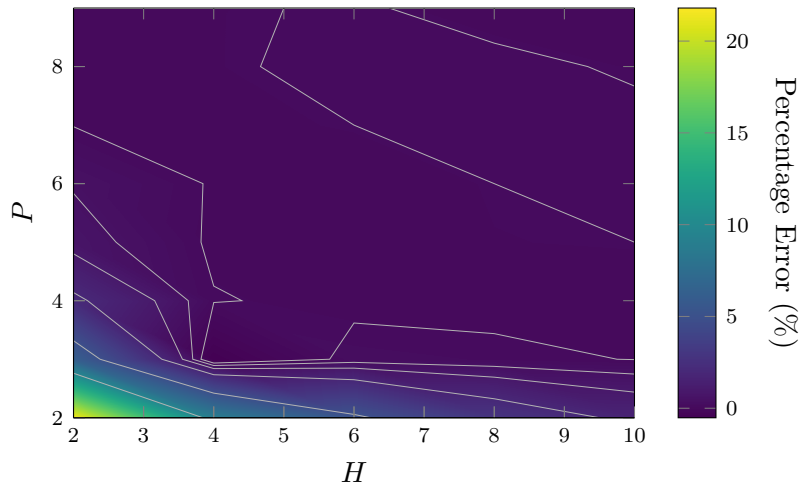
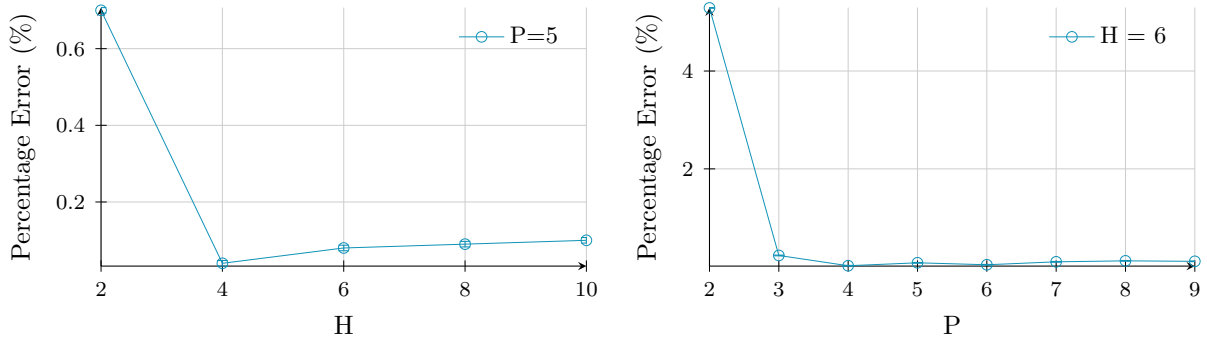


Figure 4.1: Conduction velocity percentage error vs H and P .

At low polynomial order and coarse mesh spacing, the error at the boundary of the action potential is high (20%). If we were to complete training on a dataset that was not refined enough in either metric, we would have a simulation that does not correspond to the biophysical morphology and the error at longer periods.

However, increasing the polynomial order and mesh spacing beyond a certain point does not increase the accuracy and can be counterproductive, increasing computational cost as well as introducing worsening Gibbs phenomena at the wavefront.

An increase in the polynomial order to $P = 5$ and mesh spacing to $H = 6$ gives a percentage error of < 0.01 . This can be seen in Figure 4.2, with little improvement available at higher orders or denser meshes.



(a) Percentage Error vs H with connected line and error bars.

(b) Percentage Error vs P for H = 6.

Figure 4.2: Comparison of percentage errors for different parameters.

4.2 Network Architecture Selection

We now want to pick a network architecture that is simultaneously small enough to run on our system (A40 GPU with 40GB RAM), while being complex enough to capture the dynamics in a short time. We run a hyperparameter optimisation using Optuna [37].

4.2.1 Hyperparameter Study Results

The hyperparameters we chose to optimise over were chosen to allow us to explore both optimal architecture and efficient learning.

In our results, we will only use models that are perceived to have 'learnt' the training data, that is, correctly predicting on the validation data with a validation loss, $\mathcal{L} < 0.2$.

As expected, more complex architectures produced models that took longer to run (see Figure ??), but that produced this also lead to improved performance, Figure ??.

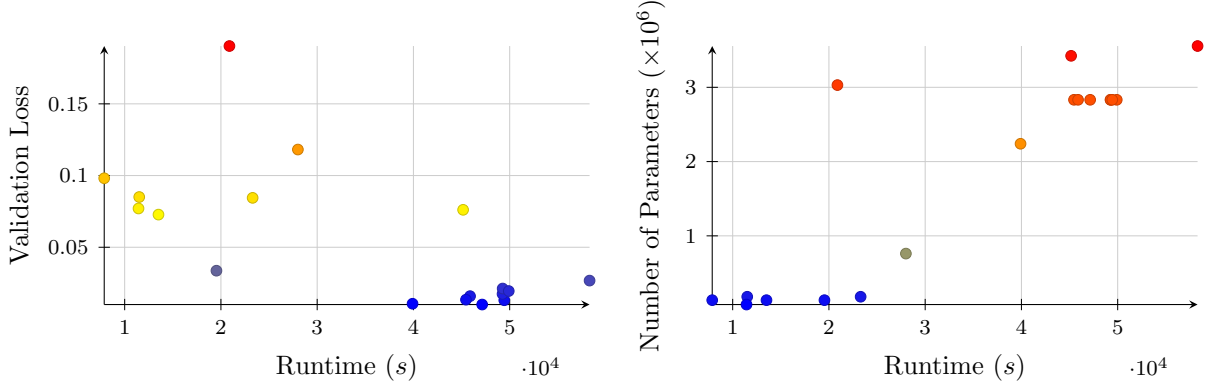


Figure 4.3: Runtime vs validation loss

However, we can also see that a well-constructed smaller architecture can produce similar results in much less time, which will allow for quicker design iteration and testing.

Our whole Optuna parameter search can be found in Figure 4.4

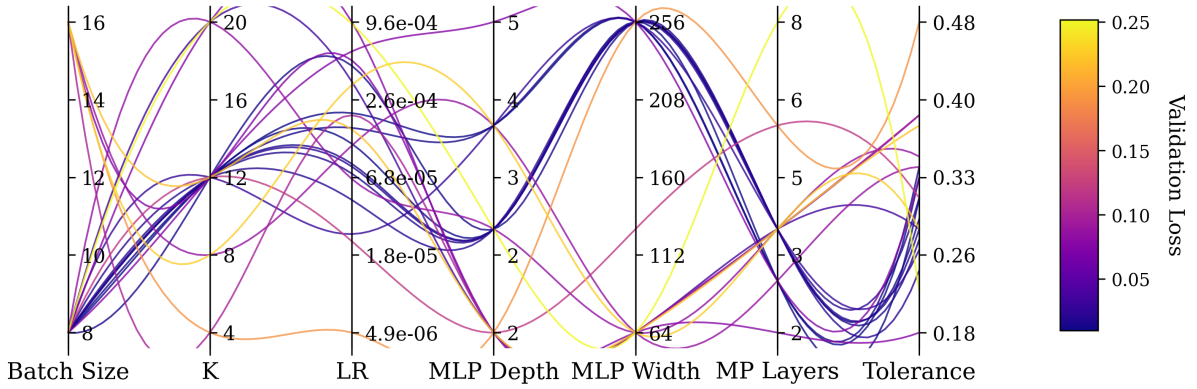


Figure 4.4: Parallel coordinate plot showing hyperparameter combinations used during optimisation, with their validation loss (Objective Value)

4.3 Comparing Loss Functions

The following section compares five loss functions: MuS, Grad, DD, Gauss and GradGauss ???. Each is devised to improve the network's ability to capture cardiac wave propagation accurately, with the motivation for each explained in the methodology. Each was trained on identical GNN architectures and the same data, with the same time constraints placed on the training time, see ??? for details. We then need to test the results, which is done through four test scenarios (standard propagation, planar wave generalisation, wave collision dynamics, and extended-domain repolarisation), each test designed to evaluate the performance of a different aspect of the action potential. The metrics used to assess will be R^2 , the coefficient of determination, and cardiac-specific metrics (quasi-dissipative/ dispersive error and action potential morphology preservation). Results are organised by metric to highlight how each loss balances wave-front

sharpness, timing fidelity, boundary enforcement, and phase-specific weighting, with statistical significance testing applied to all comparisons.

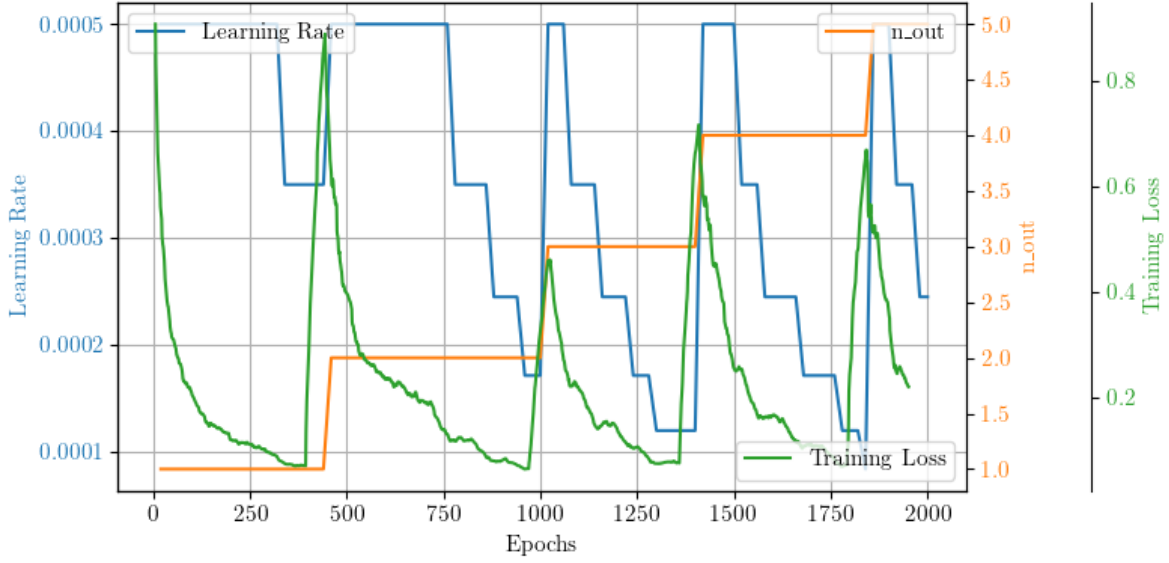


Figure 4.5: Caption

4.3.1 Relative Loss Results

First, we focus on the domain as a whole using the coefficient of determination, the results can be seen in Figure 4.6. The first thing to note is that the DD loss is performing below the standard of the other loss functions, producing much more error. Next, we note that the Grad, Gauss and GradGauss functions outperform the MuS loss in all test scenarios, suggesting improvements in the ability to learn the diffusion-reaction phenomena. The performance on the collision dataset is similar across the best three, with the GradGuass loss providing a slight improvement. In the long time simulation, the Gauss loss performs better than Grad and GradGauss, suggesting that the wavefront focus of the Gauss loss reduces errors at the wavefront, which compounded the most over time. This leads nicely on to looking at the wavefront metric, to see that our hypothesis is supported

We now see in Figure 4.7, that the trends we observed in our first measurements are repeated here. First, the DD loss again produces worse results, with higher error at the wavefront in every scenario. We see again the MuS loss is outperformed by the three remaining losses, supporting our hypotheses that a loss function that encourages the network to focus on the wavefront produces a model that better predicts conduction velocity. The last result that is of note is that again, the Gauss loss performs best in long time scenarios, which supports the hypothesis that the gaussian mask applied during the loss function also forces the network to learn the repolarisation rate.

It should be noted that the wavefront metric is measured by taking the MSE only on points at the wavefront, for which we gave a transmembrane potential of -35 mV. This is of note as in the longer time simulations, the repolarising tissue starts to reach this -35 mV transmembrane

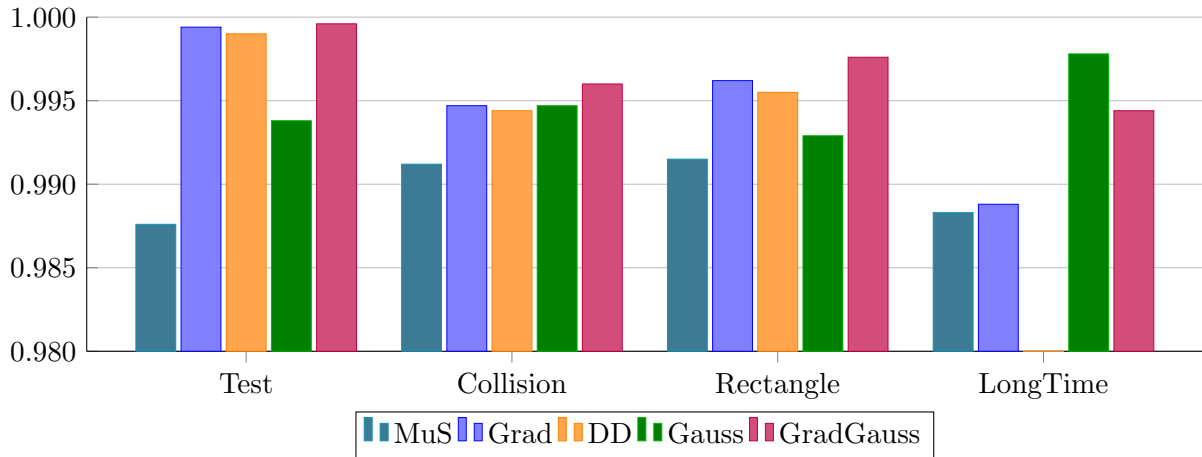


Figure 4.6: Coefficient of Determination, R^2 , for the loss functions over the range of test scenarios. (Higher is better)

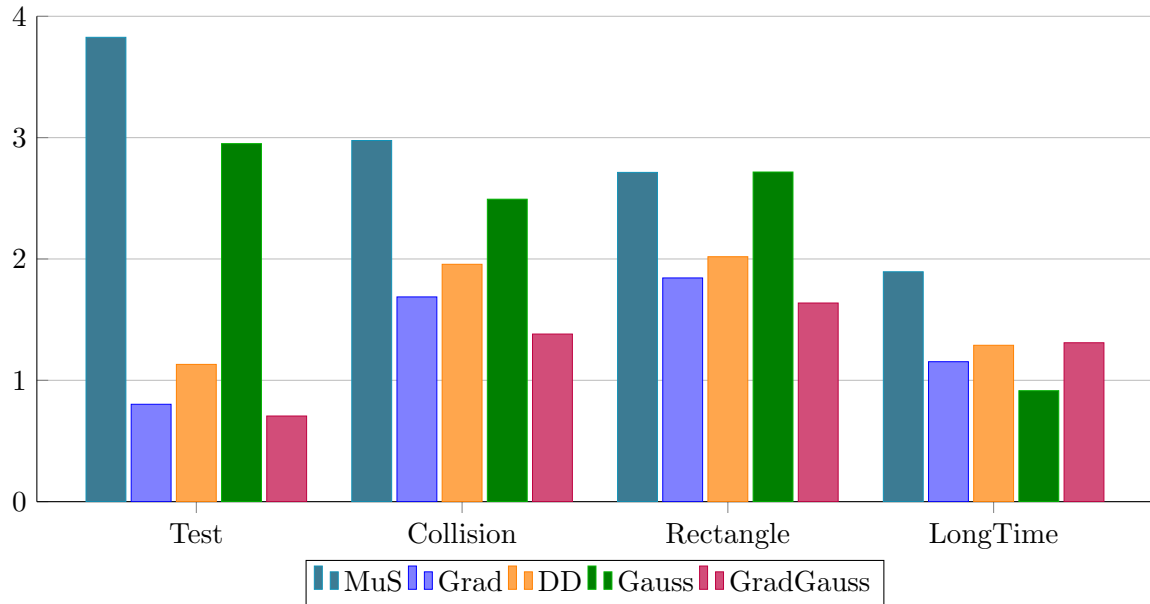


Figure 4.7: Loss function performance, measured using our Wavefront metric. (Lower is better)

potential. Therefore, our wavefront metric is now simultaneously measuring error in the repolarisation rate and wavefront/conduction velocity. This means that we will need to inspect the transmembrane potential field to decouple these phenomena and infer the network's properties.

TODO: TMP Fields to show above results???

TODO: REMOVE EXTRA METRICS????

Diss/Disp Split

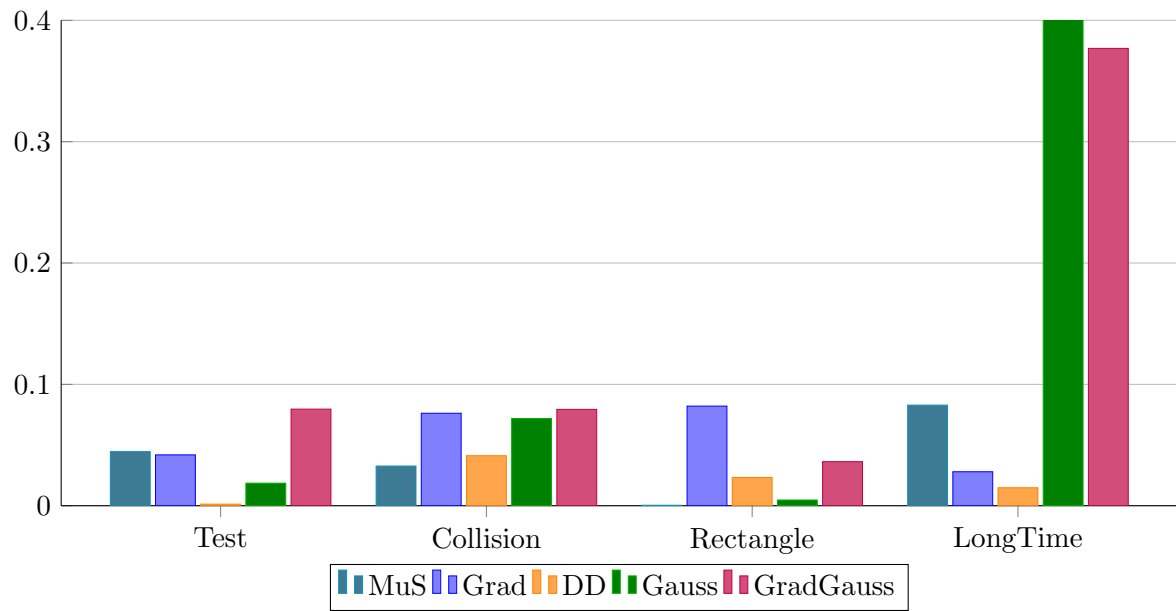


Figure 4.8: Caption

Edge Error

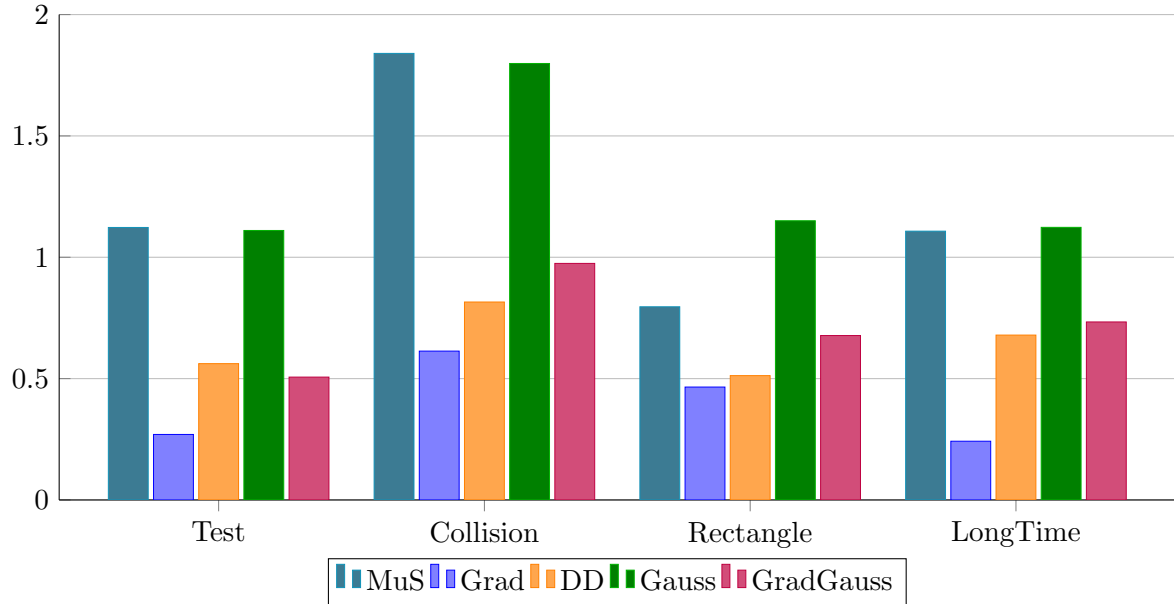


Figure 4.9: Caption

4.4 Integrated Model Configuration: AP-GNN

TODO: Maybe a little explanation about set-up and training

4.4.1 Training Performance

4.5 Integrated Model Configuration: AP-GNN-STIM

To do this, we need to be able to convey when, where and for how long a stimulus is going to occur. We have implemented this by adding a field to the node attribute, `stim_mask`, which will have a value between 0 and 1, where 0 indicates not stimulated and values greater than 0 indicate the distance from the centre of the stimulus.

4.5.1 Training Performance

Chapter 5

Model Performance: Results and Analysis

This section contains the results for the final models, discussing performance capturing AP propagation, and what section of the AP the models struggle with.

5.1 Network Performance: AP-GNN

First we evaluate our model’s performance on our test datasets. The model accurately captures the wavefront morphology and generalises well to planar waves, with minimal error produced on the Rect data.

Table 5.1: Performance of the network evaluated on different datasets.

Dataset	R^2	MSE	Wavefront Error
DS1	0.9999	0.0442	0.1894
Rect	0.9999	0.0490	0.2315
Collision	0.9983	1.7678	0.4941
LongTime	0.9905	5.9706	0.7254

Looking at the error in the collision test, we notice a small increase in the error. This is due to a small error introduced at the wavefront at the edges of the collision, as seen in our error plots in Figure 5.1. Also to note is that the transmembrane potential does not spike as much. However, the model achieves a good accuracy, picking up the most important features.

The next test we look at is the models performance over longer time periods. As seen in Figure 5.2, the model accurately captures the wavefront, however, the tissue that should be repolarising undergoes a secondary stimulus-like effect. This is likely due to some instability in our network, causing the model to struggle to capture repolarisation over longer time frames.

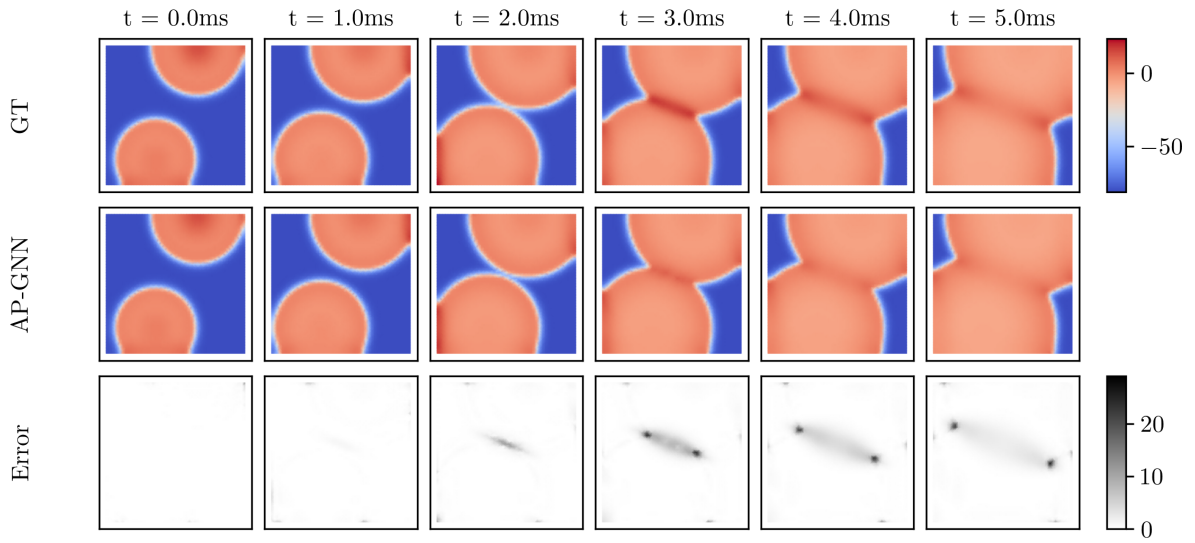


Figure 5.1: Inference of AP-GNN on the Collision dataset. Ground truth, prediction and error

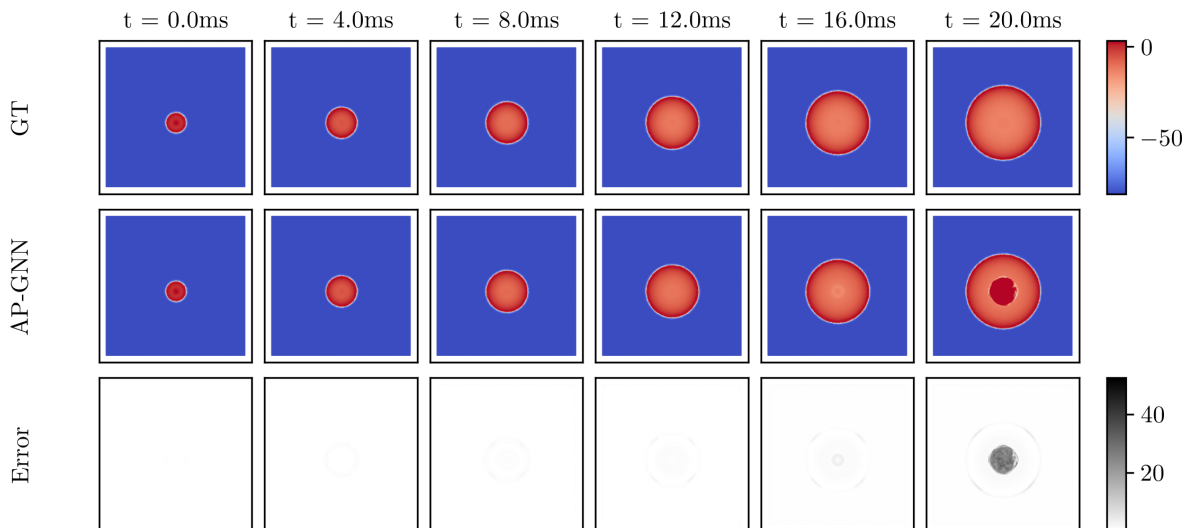


Figure 5.2: AP-GNN inference on long time dataset

TODO: ADD mV to colourbars

This can be further seen in Figure 5.3, where the action potential is well matched to a point, but at any point the tissue reaches, $V_m = -5$ – -15 mV, the inference becomes unstable.

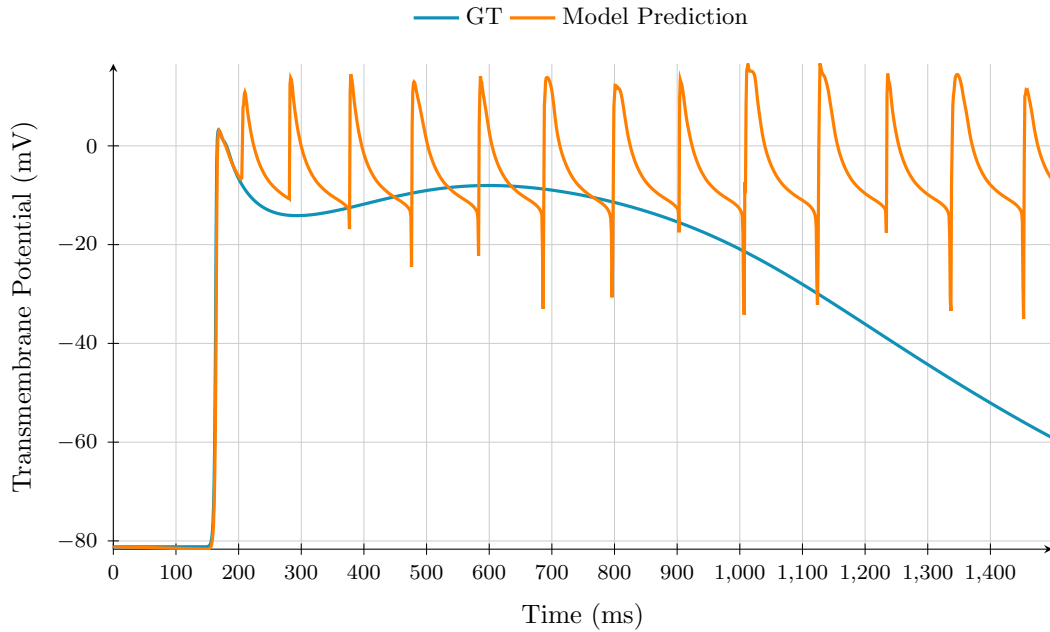


Figure 5.3: Inference of action potential over 1.5 seconds by AP-GNN.

The culmination of this model is to motivate the generation of our next model, with the express goal of capturing stimuli and correctly capturing the repolarisation phase of the action potential.

5.2 AP-GNN-STIM

With this goal in mind we shall now evaluate the success of this model against the same test set for short times to verify performance similar to that of AP-GNN in short time.

Table 5.2: Performance of the network evaluated on different datasets.

Dataset	R^2	MSE	Wavefront Error
DS1	0.	0.	0.
Rect	0.	0.	0.
Collision	0.	0.	0.
LongTime	0.	0.	0.

TODO: Finish Results here

Figure 5.5: Action potnetial for AP-GNN-STIM.

5.2.1 Stimuli Capture

Now we will see the ability of AP-GNN-STIM to capture the evolution of stimuli

Figure 5.4: AP-GNN-STIM predicting the evolution of a new stimulus, with the stimulus mask passed.

Rect stim???

We see that the model is able to generalise to stimuli with non-circular shapes.

5.2.2 AP Graph

5.3 Comparing to Nektar++ to solve

Timing of the data process

The inference was completed using a system with an Intel Xenon 6240R CPU @ 2.40GHz and an Nvidia A40 GPU, while Nektar++ simulations were completed on a 16 core CPU.

Table 5.3: Performance of the network evaluated on different datasets.

Time (s)	GMSH	NEKTAR++	CHK to VTU	VTU 2 H5
Simulation	0.01	23658	13336	39947
Inference				

5.4 Testing on Non-Linear Domains

Our model is trained on data generated on a linear mesh, so it has regular edge lengths. Therefore, we will test the generalisation on coarser non-linear meshes, MedMesh and CoarseMesh.

As seen in Figure 5.6, our non-linear meshes are coarser; this mimics the mesh resolution that can be expected clinically [3].

First in Figure 5.6, we see our models performance on when infering on the mesh it was trained on. Then in Figure 5.8 and Figure 5.9, we see that the wavefront is distorted, with the model unable to generalise to different mesh shapes and resolutions.

TODO: Probably remove fig and reference earlier figure.

This suggests that in future work, more thought needs to be put into generating a wider range of meshes for our training data. This could also signal a directional bias exhibited by our

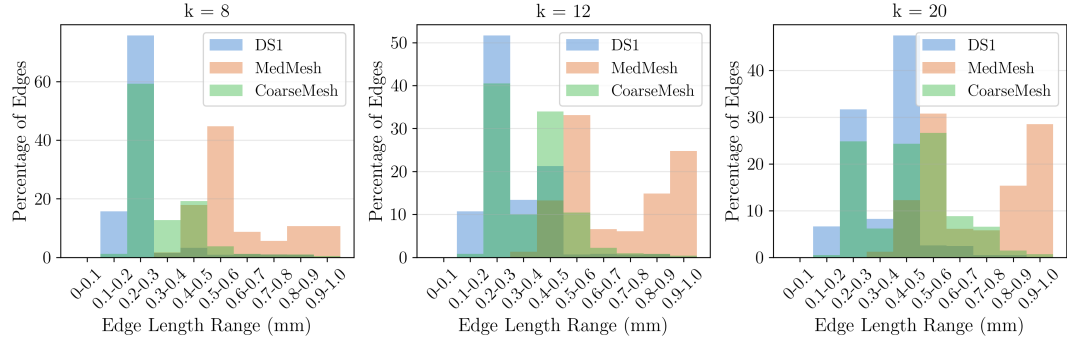


Figure 5.6: Distribution of edge lengths for the meshes, DS1, MedMesh and CoarseMesh

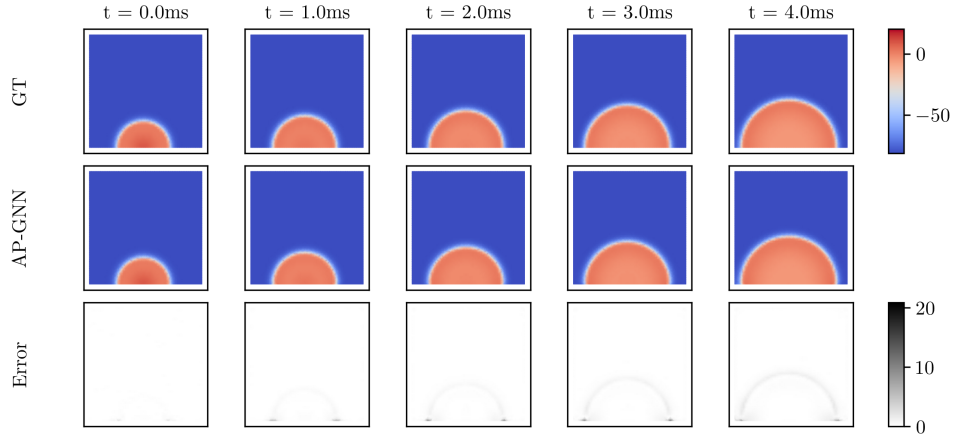


Figure 5.7: Performance of validation data.

model; this could lead us to look at new GNN architectures, [4].

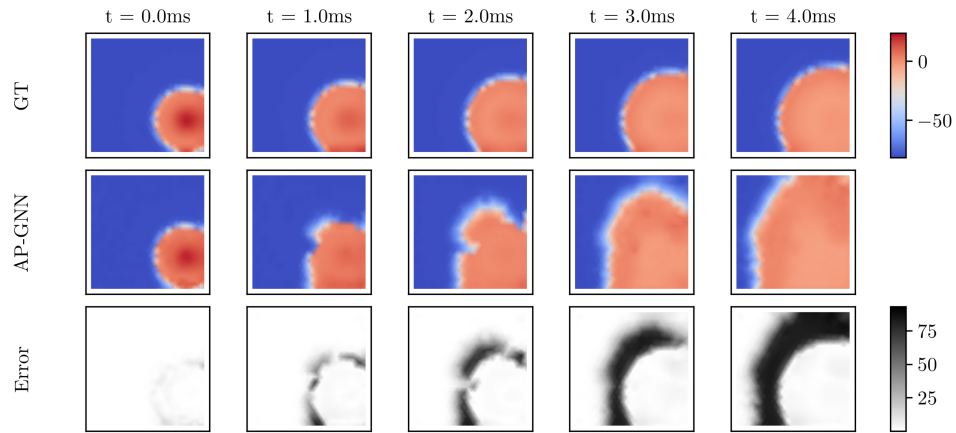


Figure 5.8: Performance on a coarser mesh, MedMesh

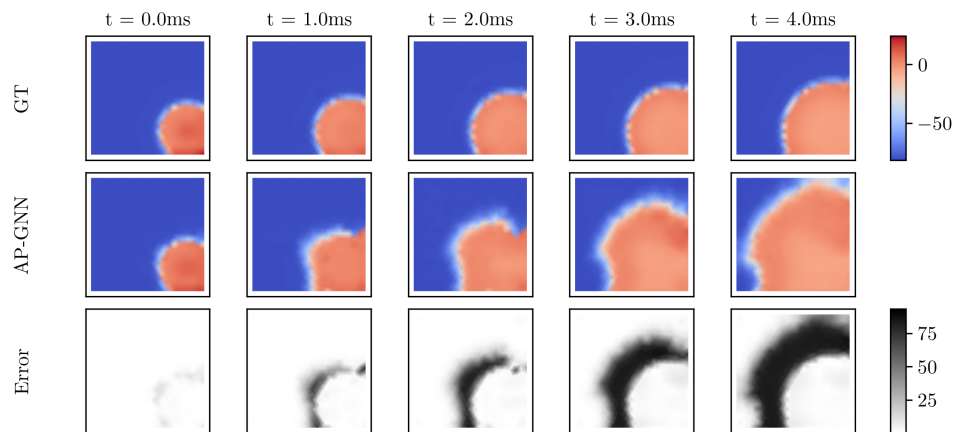


Figure 5.9: Performance on a coarser mesh, CoarseMesh

TODO: Complete section

5.5 Limitations and Further Study

Edge Size

Tissue Health/Characteristics

Multiscale temporal behaviour

Fast depolarisation, slow repolarisation.

Additional Fields Passing??

Chapter 6

Conclusions

We have designed two, single-scale graph neural networks frameworks, AP-GNN and AP-GNN-STIM, with the intension of predicting the action potential propagation in cardiac tissue. We aimed to produce physically consistent conduction velcity and repolaristation rate at the two most clinically relavent phenomena. The networks were trained using simulations produced on simple 2D regular meshes using Nektar++.

AP-GNN displayed an ability to predict complex and unseen wave dynamics, such as collisions. The inference of AP-GNN, was completed in timme frame of minutes where simulations took hours to days to complete, showing a significant reduction in the time frame required.

We then went on to attempt to capture multiple stimuli and the evolution of stimuli utilising, AP-GNN-STIM. The model was capable of capturing the short term dynamics of stimuli, however struggled to capture the longer time evolutions, restricting the ability for the model to capture propagation on a physically relevant time scale (i.e. seconds).

The framework allows for multi-scale graphs to be built and could be a good avenue for further investigation.

Bibliography

- [1] R. H. Clayton et al. “Models of cardiac tissue electrophysiology: Progress, challenges and open questions”. In: *Progress in Biophysics and Molecular Biology* 104.1-3 (Jan. 2011), pp. 22–48. ISSN: 0079-6107. DOI: 10.1016/J.PBIOMOLBIO.2010.05.008. URL: <https://www.sciencedirect.com/science/article/pii/S0079610710000362>.
- [2] Atul Verma et al. “Approaches to Catheter Ablation for Persistent Atrial Fibrillation”. In: *New England Journal of Medicine* 372.19 (May 2015), pp. 1812–1822. ISSN: 0028-4793. DOI: 10.1056/NEJMOA1408288/SUPPL{_}FILE/NEJMOA1408288{_}DISCLOSURES.PDF. URL: <https://www.nejm.org/doi/pdf/10.1056/NEJMoa1408288>.
- [3] Olaf Doessel et al. “Personalized Cardiac Computational Models: From Clinical Data to Simulation of Infarct-Related Ventricular Tachycardia”. In: *Frontiers in Physiology* — www.frontiersin.org 1 (2019), p. 580. DOI: 10.3389/fphys.2019.00580. URL: www.frontiersin.org.
- [4] Mario Lino et al. “REMuS-GNN: A Rotation-Equivariant Model for Simulating Continuum Dynamics”. In: *Proceedings of Machine Learning Research* 196 (May 2022), pp. 226–236. ISSN: 26403498. URL: <https://arxiv.org/pdf/2205.07852>.
- [5] *The Heart Conduction System - QCG*. URL: <https://qcg.com.au/healthy-heart/conduction-system/>.
- [6] Hashem A. Dbouk et al. “Connexins: a myriad of functions extending beyond assembly of gap junction channels”. In: *Cell Communication and Signaling : CCS* 7 (2009), p. 4. ISSN: 1478811X. DOI: 10.1186/1478-811X-7-4. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC2660342/>.
- [7] Darren A. Hooks et al. “Laminar arrangement of ventricular myocytes influences electrical behavior of the heart”. In: *Circulation Research* 101.10 (Nov. 2007). ISSN: 00097330. DOI: 10.1161/CIRCRESAHA.107.161075,. URL: <https://www.ahajournals.org/doi/pdf/10.1161/CIRCRESAHA.107.161075?download=true>.
- [8] Robert J. Young and Alexander V. Panfilov. “Anisotropy of wave propagation in the heart can be modeled by a Riemannian electrophysiological metric”. In: *Proceedings of the National Academy of Sciences of the United States of America* 107.34 (Aug. 2010), pp. 15063–15068. ISSN: 00278424. DOI: 10.1073/PNAS.1008837107/-/DCSUPPLEMENTAL. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC2930580/>.

- [9] Mengyun Qiao et al. “A personalized time-resolved 3D mesh generative model for unveiling normal heart dynamics”. In: *Nature Machine Intelligence* 7.5 (May 2025), pp. 800–811. ISSN: 25225839. DOI: 10.1038/S42256-025-01035-5; SUBJMETA=1628, 1647, 245, 4019, 592, 631, 692, 75; KWRD=CARDIOVASCULAR+DISEASES, MAGNETIC+RESONANCE+IMAGING. URL: <https://www.nature.com/articles/s42256-025-01035-5>.
- [10] Fanwei Kong and Shawn C. Shadden. “Learning Whole Heart Mesh Generation From Patient Images for Computational Simulations”. In: *IEEE Transactions on Medical Imaging* 42.2 (Feb. 2023), pp. 533–545. ISSN: 1558254X. DOI: 10.1109/TMI.2022.3219284,. URL: <https://pubmed.ncbi.nlm.nih.gov/36327186/>.
- [11] Xulei Qin and Baowei Fei. “DTI template-based estimation of cardiac fiber orientations from 3D ultrasound”. In: *Medical Physics* 42.6 (June 2015), p. 2915. ISSN: 24734209. DOI: 10.1118/1.4921121. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC4441706/>.
- [12] Shana R. Watson, James D. Dormer, and Baowei Fei. “Imaging technologies for cardiac fiber and heart failure: a review”. In: *Heart failure reviews* 23.2 (Mar. 2018), p. 273. ISSN: 15737322. DOI: 10.1007/S10741-018-9684-1. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC6121700/>.
- [13] Alejandro Lopez-Perez, Rafael Sebastian, and Jose M. Ferrero. “Three-dimensional cardiac computational modelling: methods, features and applications”. In: *BioMedical Engineering OnLine* 14.1 (Apr. 2015), p. 35. ISSN: 1475925X. DOI: 10.1186/S12938-015-0033-5. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC4424572/>.
- [14] Hugh D. Young and Rodger A. Freedman. *Sears and Zemansky’s University Physics with Modern Physics, Global Edition*. 15th ed. Pearson, Oct. 2024.
- [15] G. T. Lines et al. “Mathematical models and numerical methods for the forward problem in cardiac electrophysiology”. In: *Computing and Visualization in Science* 5.4 (2003), pp. 215–239. ISSN: 14330369. DOI: 10.1007/S00791-003-0101-4/METRICS. URL: <https://link.springer.com/article/10.1007/s00791-003-0101-4>.
- [16] Mark Potse et al. “A comparison of monodomain and bidomain reaction-diffusion models for action potential propagation in the human heart”. In: *IEEE Transactions on Biomedical Engineering* 53.12 (2006), pp. 2425–2435. ISSN: 00189294. DOI: 10.1109/TBME.2006.880875,. URL: <https://pubmed.ncbi.nlm.nih.gov/17153199/>.
- [17] *Nektar++: Spectral/hp Element Framework*. URL: <https://doc.nektar.info/userguide/latest/>.
- [18] Marc Courtemanche, Rafael J. Ramirez, and Stanley Nattel. “Ionic mechanisms underlying human atrial action potential properties: Insights from a mathematical model”. In: *American Journal of Physiology - Heart and Circulatory Physiology* 275.1 44-1 (1998). ISSN: 03636135. DOI: 10.1152/AJPHEART.1998.275.1.H301,. URL: <https://pubmed.ncbi.nlm.nih.gov/9688927/>.
- [19] Rubin R. Aliev and Alexander V. Panfilov. “A Simple Two-variable Model of Cardiac Excitation”. In: *Chaos, Solitons and Fractals* 7.3 (1996), pp. 293–301. ISSN: 09600779. DOI: 10.1016/0960-0779(95)00089-5.

- [20] C. H. Luo and Y. Rudy. “A model of the ventricular cardiac action potential. Depolarization, repolarization, and their interaction”. In: *Circulation Research* 68.6 (1991), pp. 1501–1526. ISSN: 00097330. DOI: 10.1161/01.RES.68.6.1501,. URL: <https://pubmed.ncbi.nlm.nih.gov/1709839/>.
- [21] K. H.W.J. Ten Tusscher and A. V. Panfilov. “Alternans and spiral breakup in a human ventricular tissue model”. In: *American Journal of Physiology - Heart and Circulatory Physiology* 291.3 (2006), pp. 1088–1100. ISSN: 03636135. DOI: 10.1152/AJPHEART.00109.2006,. URL: <https://journals.physiology.org/doi/pdf/10.1152/ajpheart.00109.2006>.
- [22] Peter W. Battaglia et al. “Relational inductive biases, deep learning, and graph networks”. In: (June 2018). URL: <https://arxiv.org/pdf/1806.01261>.
- [23] Shai Shalev-Shwartz and Shai Ben-David. “Neural Networks”. In: *Understanding Machine Learning* (May 2014), pp. 228–242. DOI: 10.1017/CB09781107298019.021. URL: <https://www.cambridge.org/core/books/understanding-machine-learning/neural-networks/DB98EE3B4CD3FE2211B4371171FBEA44>.
- [24] *torch.nn — PyTorch 2.8 documentation*. URL: <https://docs.pytorch.org/docs/stable/nn.html>.
- [25] Jurafsky Daniel and James H Martin. “Speech and Language Processing”. In: (2024).
- [26] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. “Learning representations by back-propagating errors”. In: *Nature* 323.6088 (1986), pp. 533–536. ISSN: 00280836. DOI: 10.1038/323533A0.
- [27] *PyTorch documentation — PyTorch 2.7 documentation*. URL: <https://docs.pytorch.org/docs/stable/index.html>.
- [28] Mario Lino et al. “Multi-scale rotation-equivariant graph neural networks for unsteady Eulerian fluid dynamics”. In: *Physics of Fluids* 34.8 (Aug. 2022). ISSN: 10897666. DOI: 10.1063/5.0097679/2847850. URL: [/aip/pof/article/34/8/087110/2847850/Multi-scale-rotation-equivariant-graph-neural](https://aip/pof/article/34/8/087110/2847850/Multi-scale-rotation-equivariant-graph-neural).
- [29] Hervé Guillard. “Node-nested multi-grid method with Delaunay coarsening”. In: (1993). DOI: 10.34894/VQ1DJA. URL: <https://inria.hal.science/inria-00074773%20https://inria.hal.science/inria-00074773/document>.
- [30] Maëlis Morier et al. “Learning Cardiac Electrophysiology with Graph Neural Networks for Fast Data-Driven Personalised Predictions”. In: (2025), pp. 138–150. ISSN: 16113349. DOI: 10.1007/978-3-031-94559-5_{13}/FIGURES/13. URL: https://link.springer.com/chapter/10.1007/978-3-031-94559-5_13.
- [31] Clara Herrero Martin et al. “EP-PINNs: Cardiac Electrophysiology Characterisation Using Physics-Informed Neural Networks”. In: *Frontiers in Cardiovascular Medicine* 8 (2021), p. 768419. ISSN: 2297055X. DOI: 10.3389/FCVM.2021.768419/FULL. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC8850959/>.
- [32] *Gmsh 4.14.0*. URL: <https://gmsh.info/doc/texinfo/gmsh.html>.

- [33] Steven A. Niederer et al. “Verification of cardiac tissue electrophysiology simulators using an N-version benchmark”. In: *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 369.1954 (Nov. 2011), pp. 4331–4351. ISSN: 1364503X. DOI: 10.1098/RSTA.2011.0139; JOURNAL: JOURNAL:RSTA1887; CTYPE: STRING: JOURNAL. URL: /doi/pdf/10.1098/rsta.2011.0139.
- [34] Shun Zhang. “Battling Gibbs phenomenon: On finite element approximations of discontinuous solutions of PDEs”. In: *Computers & Mathematics with Applications* 122 (Sept. 2022), pp. 35–47. ISSN: 0898-1221. DOI: 10.1016/J.CAMWA.2022.07.014. URL: <https://www.sciencedirect.com/science/article/pii/S0898122122002978>.
- [35] *ParaView - Open-source, multi-platform data analysis and visualization application*. URL: <https://www.paraview.org/>.
- [36] *mariolinov / mus-gnn_datasets — Bitbucket*. URL: https://bitbucket.org/mariolinov/mus-gnn_datasets/src/master/.
- [37] Takuya Akiba et al. “Optuna: A Next-generation Hyperparameter Optimization Framework”. In: (July 2019).
- [38] Shuhei Watanabe. “Tree-Structured Parzen Estimator: Understanding Its Algorithm Components and Their Roles for Better Empirical Performance”. In: (Apr. 2023). URL: <https://arxiv.org/pdf/2304.11127>.
- [39] Indu Kant Deo and Rajeev K Jaiman. “Harnessing Loss Decomposition for Long-Horizon Wave Predictions via Deep Neural Networks”. In: (Dec. 2024). URL: <https://arxiv.org/pdf/2412.02924>.

Appendix A

A.1 Bidomain Equation Derivation

Deriving the governing equations for our Bidomain model.

From

$$\nabla \cdot \mathbf{J}_i = I_m \quad (\text{A.1})$$

$$\nabla \cdot \mathbf{G}_i(\nabla \phi_i) = \beta_m \left(C_m \frac{dV_m}{dt} + i_{ion} \right) \quad (\text{A.2})$$

Now using $\phi_i = V_m + \phi_e$

$$\nabla \cdot \mathbf{G}_i(\nabla V_m + \nabla \phi_e) = \beta_m \left(C_m \frac{dV_m}{dt} + i_{ion} \right) \quad (\text{A.3})$$

Now from,

$$\nabla \cdot (\mathbf{J}_i + \mathbf{J}_e) = 0 \quad (\text{A.4})$$

Again using $\phi_i = V_m + \phi_e$,

$$\nabla \cdot \left((\mathbf{G}_i + \mathbf{G}_e) \nabla \phi_e + \mathbf{G}_i \nabla V_m \right) = 0 \quad (\text{A.5})$$

which, when rearranged yields,

$$\nabla \cdot ((\mathbf{G}_i + \mathbf{G}_e) \nabla \phi_e) = -\nabla \cdot (\mathbf{G}_i \nabla V_m) \quad (\text{A.6})$$

A.2 Monodomain Equation Derivation

Deriving Equations for our Monodomain model from our governing equation for the bidomain model.

We with our assumption,

$$\mathbf{G}_e = \lambda \mathbf{G}_i \quad (\text{A.7})$$

Then equation A.6 can be expressed as,

$$\nabla \cdot ((\lambda + 1) \mathbf{G}_i \nabla \phi_e) = -\nabla \cdot (\mathbf{G}_i \nabla V_m) \quad (\text{A.8})$$

Substituting this into equation A.3 gives,

$$\nabla \cdot (\mathbf{G}_i \nabla V_m) - \nabla \cdot \left(\frac{1}{\lambda + 1} \mathbf{G}_i \nabla V_m \right) = \beta_m \left(C_m \frac{dV_m}{dt} + i_{ion} \right) \quad (\text{A.9})$$

Now we can define the effective conductivity as, $\mathbf{G} = \frac{\lambda}{\lambda+1} \mathbf{G}_i$, which gives us the final form of our monodomain equation as,

$$\nabla \cdot \mathbf{G} \nabla V_m = \beta_m \left(C_m \frac{dV_m}{dt} + i_{ion} \right) \quad (\text{A.10})$$

Appendix B

B.1 Mesh Production

Gmsh code for producing a rectangular mesh with height L_y , width L_x and a resolution of RES cells per mm [32].

```
1 width=Lx;
2 height=Ly;
3 res = RES;
4
5 Point(1) = {-width/2,-height/2,0,0.1};
6 l[] = Extrude {width,0,0} {
7     Point{1}; Layers{width*res}; Recombine;
8 };
9 s[] = Extrude {0,height,0} {
10     Line{l[1]}; Layers{height*res}; Recombine;
11 };
12 Physical Surface(0) = {s[1]};
13 Physical Line(1) = {1,2,3,4};
```

B.2 *CardiacDataset* Class

```
1     a
```

Appendix C

C.1 Message Passing (MP) Algorithm

Algorithm 1 Full GN Block Computation

Require: Graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, u)$

Ensure: Updated graph with $\mathcal{V}', \mathcal{E}', u'$

```

1: Edge Update:
2: for  $k \in \{1, \dots, N^e\}$  do                                ▷ For each edge
3:    $e'_k \leftarrow \phi^e(e_k, v_{r_k}, v_{s_k}, u)$                 ▷ Update edge attributes
4: end for
5: Node Update:
6: for  $i \in \{1, \dots, N^v\}$  do                                ▷ For each node
7:    $\bar{e}'_i \leftarrow \{e'_k \mid r_k = i \text{ or } s_k = i\}_{k=1}^{N^e}$   ▷ Aggregate edge attr. per node
8:    $\bar{e}''_i \leftarrow \rho^{e \rightarrow v}(\bar{e}'_i)$                     ▷ Aggregate edge attributes
9:    $v'_i \leftarrow \phi^v(\bar{e}''_i, v_i, u)$                       ▷ Update node attributes
10: end for
11: Global Update:
12:  $\mathcal{V}' \leftarrow \{v'_i\}_{i=1}^{N^v}$ 
13:  $\mathcal{E}' \leftarrow \{(e'_k, r_k, s_k)\}_{k=1}^{N^e}$ 
14:  $\bar{e}' \leftarrow \rho^{e \rightarrow u}(\mathcal{E}')$                         ▷ Aggregate edge attr. globally
15:  $\bar{v}' \leftarrow \rho^{v \rightarrow u}(\mathcal{V}')$                         ▷ Aggregate node attr.
16:  $u' \leftarrow \phi^u(\bar{e}', \bar{v}', u)$                           ▷ Update global attributes
17: return  $(u', \mathcal{V}', \mathcal{E}')$ 

```

[22]

Algorithm 2 GN Block Pseudo Code

```

Edge Update:  $e'_k \leftarrow \phi^e(e_k, v_{r_k}, v_{s_k}, u) \forall k$ 
Node Update:  $v'_i \leftarrow \phi^v(\rho^{e \rightarrow v}(\bar{e}'_i), v_i, u) \forall i$ 
Global Update:  $u' \leftarrow \phi^u(\rho^{e \rightarrow u}(\mathcal{E}'), \rho^{v \rightarrow u}(\mathcal{V}'), u)$ 
Return:  $(u', \mathcal{V}', \mathcal{E}')$ 

```
